HEC MONTREAL

2026

AI-Powered Tourism Intelligence Pipeline for Destination Québec cité:

from

[Gilles Ghoussoub]

(Specialization Data Science and Business Analytics)

Abstract

This project describes the development and deployment of an automated media monitoring intelligence pipeline, built for Destination Québec cité (DQc), the organization responsible for promoting tourism across the Québec Capitale-Nationale region. The idea behind it is simple enough: take the daily flood of news coverage about Quebec City and turn it into structured, usable data that the team can actually do something with.

Every day, the pipeline collects articles from over thirty francophone and anglophone media sitemaps, plus a supplementary Google News search layer. Each article goes through a nine-stage processing workflow. The system starts by resolving URLs and scraping full article text using a headless browser set up to handle dynamic pages. From there, a filtering stage checks whether each article is genuinely relevant to the region, using rubric built around the specific municipalities, landmarks, and tourism assets that matter to DQc. Anything off-topic gets dropped early.

The articles that make it through are then enriched in several ways. The pipeline scores sentiment, assigns standardized news categories, and tags each piece with domain specific labels covering attractions, hotels, and restaurants. It also groups related articles into clusters based on how similar they are in meaning, which makes it possible to follow a single story as it spreads across different outlets. At the end of the chain, each article gets a short summary along with geographic tags and an assessment of how the coverage might shape visitor perception.

Everything the pipeline produces is stored in a cloud data warehouse and feeds into a monitoring Looker dashboard. The whole system runs daily in a containerized cloud environment with no manual intervention needed. In practice, it gives DQc a continuously updated picture of how Quebec City's tourism sector is being covered in the press, helping the team spot emerging stories, track public sentiment, and get ahead of both opportunities and risks.

Keywords: Natural Language Processing, Tourism Intelligence, News Monitoring, Data Pipeline, Cloud Computing, Sentiment Analysis, Text Classification, Web Scraping

Table of Contents

Introduction

The Organization: Destination Québec cité

This supervised project was carried out within Destination Québec cité (DQc), the official organization responsible for promoting tourism in the Capitale-Nationale region of Quebec. DQc acts as the bridge between public authorities, hotels, restaurants, attractions, event organizers, and other hospitality businesses that together make Quebec City a major travel destination. Its work covers four main areas: communications and branding, event promotion, relationships with member businesses, and data driven support for tourism planning and investment.

DQc's territory goes well beyond Quebec City itself. It covers the full city agglomeration, including boroughs like La Cité-Limoilou, Sainte-Foy–Sillery–Cap-Rouge, Beauport, and Charlesbourg, along with nearby municipalities such as L'Ancienne-Lorette, Saint-Augustin-de-Desmaures, and the Indigenous community of Wendake*. (see Figure 1).* It also takes in four surrounding regional county municipalities (MRCs): La Jacques-Cartier (ski resorts, Village Vacances Valcartier), La Côte-de-Beaupré (Mont-Sainte-Anne, Montmorency Falls, Canyon Sainte-Anne), L'Île-d'Orléans (vineyards, cideries, heritage farms), and Portneuf (Vallée Bras-du-Nord, the Chemin du Roy). The region pulls in billions of dollars in tourism revenue and welcomes millions of visitors each year, which makes keeping a close eye on media coverage and public perception a real priority for the organization.

When this project started, DQc was still handling media monitoring mostly by hand. A small team of analysts would scan news sites, Google Alerts, and social media each morning looking for articles that touched on the region's tourism interests. DQc also had partnerships with external news gathering services such as Référence Média and others, which provided curated media clippings for a fee. However, even with these paid services, the actual work still relied heavily on manual searches by employees, and the scope of what was being captured remained too narrow. The regional media landscape had grown to include dozens of digital-only publications, multilingual national outlets, and international travel media, all publishing around the clock. With no centralized system to collect, organize, and analyze all

that content, important articles were regularly being missed, emerging trends went unnoticed until they turned into problems, and DQc's leadership had no reliable, data-backed full picture of how Quebec City was actually being portrayed in the press.

Project Mandate

The mandate for this supervised project was defined in collaboration with DQc's management team. The core objective was to design, build, and deploy an artificial intelligence system capable of automating DQc's media intelligence workflow, replacing the manual, ad hoc process with a structured, repeatable, and scalable pipeline that would operate continuously with minimal human intervention. Specifically, the mandate comprised the following deliverables:

1. **Automated Data Collection:** Build a system to automatically discover and collect news articles from the web, relevant to Quebec City's tourism sector from multiple francophone and anglophone media sources on a daily basis, covering local, regional, provincial, national, and international outlets.
2. **Relevance Filtering:** Develop an intelligent filtering mechanism that can distinguish articles with genuine impact on Quebec City's tourism economy from the vast majority of general news that merely happens to mention the word "Québec," accounting for the critical ambiguity between Quebec-the-city and Quebec-the-province.
3. **NLP Analysis:** Apply natural language processing techniques to classify articles, assess their sentiment, identify semantic clusters of related coverage, and assign domain specific tourism labels (attractions, lodging, restaurants).
4. **Summarization and Perception Assessment:** Generate concise, human-readable summaries of each article along with an assessment of its impact on visitor perception of Quebec City as a destination.
5. **Data Infrastructure:** Persist all analytical outputs in a cloud data warehouse (Google BigQuery) to serve as the foundation for a real time monitoring dashboard accessible to DQc's management team and analysts via Looker.

6. **Deployment:** Containerize and deploy the pipeline on Google Cloud Platform for automated, scheduled execution without ongoing technical intervention.

The strategic objective behind this mandate was to reinforce DQc's analytical and anticipatory capabilities by automating its sectoral media monitoring, thereby enabling faster, better-informed decisions in matters of communication strategy, event planning, member engagement, and crisis response. The project was explicitly positioned not as a research prototype but as a production system intended for daily operational use by non technical staff.

Literature Review

What follows is a review of the research, tools, and models that shaped how the pipeline was built. For each one, I've tried to briefly explain not just what it is, but why I ended up choosing it over the alternatives. (The explanations in the literature review will be again reiterated further in the project as I try to explain the methods chosen in their specific project sections.)

Web Scraping: Legal and ethical framework

The legal landscape of scraping : Brown et al. (2024) provide the most comprehensive post-2020 analysis of the field, establishing that scraping implicates several overlapping areas of law simultaneously: contractual restrictions through terms of service, statutory laws governing computer access and intellectual property, and privacy regimes that vary across jurisdictions. There is no single legal standard that applies uniformly. Legal risk varies along a spectrum determined primarily by whether target content is publicly accessible without authentication, whether personal information is collected, and whether collection causes material harm to the platform. Courts have found in cases such as hiQ Labs v. LinkedIn that the Computer Fraud and Abuse Act does not apply where no technical authorization barrier is bypassed, meaning publicly accessible websites carry significantly lower legal risk than password-protected ones.

What is permissible: Krotov & Silva (2018) translate this abstract exposure into concrete operational guidance. Websites do not automatically own data generated by their users, so user-produced content carries different legal status than content the site itself created. Ideas cannot be copyrighted, only the specific form in which they are expressed. So making analysis and summarization of scraped content (which I ended up doing in this project) is safer than direct reproduction (copy and pasting). Fair use further permits limited use of copyrighted material for "non-commercial" analytical purposes. On terms of service, a prohibition on crawling is not automatically enforceable unless the user explicitly accepts those terms through a deliberate action such as account creation or checking a box. Public-facing pages with no login requirement therefore carry substantially lower contractual risk.

What creates genuine exposure: Three risks are clear across both papers. Scraping and republishing explicitly copyrighted content can constitute infringement, particularly where it displaces the original. Accessing paywalled or protected content and continuing after receiving

a cease-and-desist can trigger CFAA liability when damages exceed $5,000. And scraping at a scale that places material, demonstrable load on servers can support a trespass to chattels claim. This enforced me to set rate limits for scraping calls to not bombard websites servers.

The ethical dimension : Both papers converge on a tension no legal framework fully resolves. Krotov & Silva surface the foundational paradox: the web was designed to be open, yet website owners simultaneously treat hosted data as a proprietary asset. Brown et al. extend this into practice, noting that even technically public data raises ethical concerns when collected outside the context users originally intended. So, to my understanding, the legal status of web scraping is still somewhat of a grey area, and every website can have its unique terms and conditions. However, based on my reading, I shared the following general rule of practice with DQc: a company can scrape publicly available news content for internal analytical use, provided that (1) no personal information is collected, (2) access does not require authentication like a paid account set up, (3) the scraping does not place material load on the target servers, and (4) the data is not redistributed commercially , meaning it cannot be sold, monetized, or shared with external users.

Article extraction with Trafilatura

Once a page has been fetched, the harder problem is pulling the article text out of a noisy HTML document filled with menus, ads, comment widgets, related-content blocks, and footers. I was personally used to scrape by parsing the HTML of a website and using a regex pattern to single out specific HTML sections I wanted to scrape. But mass scraping at scale from different websites is another story as every website has a unique HTML structure and creating a Python code on my own that can handle that seemed tedious.

The pipeline uses Trafilatura *(N°1, Barbaresi, 2021)* for this. Trafilatura is a Python library purpose-built for isolating main article text from boilerplate. It works by combining XPath-based content delimitation with a cascade of fallback heuristics drawn from readability-lxml and jusText, then applying length and link-density checks to keep only the clean text and structural metadata. On Barbaresi's own benchmark of 500 documents (mostly news and blog articles in German, with samples in English, French, Chinese and Arabic), Trafilatura reaches an F-score of 0.912 (0.896 in fast mode) for correct article content extraction, beating every

other open-source extractor tested: readability-lxml at 0.804, news-please at 0.808, dragnet at 0.783, goose3 at 0.767, newspaper3k at 0.708, and jusText at 0.699. Barbaresi also points to corroborating external benchmarks, including the ScrapingHub article extraction benchmark, where Trafilatura is the most efficient open-source library, and the multilingual DANIEL corpus, where it has the best overall macro-mean. That multilingual robustness is what made it the natural choice for a bilingual Quebec media context, where the same site can mix French and English articles within a single feed.

Browser automation with Playwright

Many modern news sites do not return their article text in the initial HTML response. Content is loaded by JavaScript after the page renders, which means a plain HTTP request returns an empty shell. To handle these cases the pipeline uses Playwright, chosen over the two main alternatives, Cypress and Selenium. The empirical evidence supporting that choice comes from *(Moń & Pańczyk, 2025)*, who run all three frameworks against the same set of test cases on an open-source e-commerce application, averaging each result over ten executions. Playwright achieved the shortest execution time in more than half of the ten test cases and the lowest CPU usage in the majority, while Selenium edged it out on RAM. On top of those numbers, Playwright supports six programming languages, all four major browsers, headless mode, parallel execution, and built-in screenshot and video capture, and has surpassed both competitors on npm downloads and GitHub stars. The features that matter most for scraping dynamic news sites, namely browser-context isolation (each scrape runs in a clean state), network interception (so the pipeline can block third-party trackers and ads), and auto-waiting (so the scraper does not race the page), all come out of the box.

Sentiment analysis backbone

In the text mining course, we used lexicon-based approaches for sentiment. However, since this project was multilingual, I tried to move away from lexicon-based or bag-of-words approaches.

Sentiment analysis in the pipeline is grounded in transformer-based multilingual models rather than lexicon-based or bag-of-words approaches, because the latter struggle with negation,

sarcasm, and the casual register that appears in news commentary. The chosen model, cardiffnlp/twitter-xlm-roberta-base-sentiment, is rooted in the TweetEval benchmark introduced by *(N°2, Barbieri et al., 2020)*. TweetEval unifies seven Twitter classification tasks (sentiment, emotion, irony, hate, offensive, stance, and emoji) under a single evaluation framework, addressing what the authors describe as a fragmented experimental landscape where every new dataset comes with its own protocol. Their main finding is that starting from existing pre-trained generic language models and continuing training on Twitter corpora consistently outperforms training from scratch on Twitter data alone. That conclusion is what supports using a domain-adapted XLM-R variant for the pipeline. Because no labeled Quebec tourism media dataset existed at the start of the project, the TweetEval benchmark served as the primary external validation signal that XLM-RoBERTa-based models provide robust bilingual French/English sentiment classification without domain-specific fine-tuning.

Aspect-based sentiment analysis

A single news article often expresses different sentiments toward different entities (positive about a hotel chain's renovation, negative about its labour practices, neutral about its acquisition). The experimental aspect-based sentiment analysis (ABSA) stage of the pipeline addresses this, and its architectural foundation comes from *(Zhao et al., 2022)*. The authors combine DeBERTa, which uses a disentangled attention mechanism that separates word content and position, with a Local Context Focus (LCF) layer that pushes the model to weight the words near the target aspect more heavily than the rest of the sentence. The result is aspect-conditional sentiment prediction: the same article text yields different polarity scores depending on which entity or topic is being assessed. The authors evaluate on the SemEval-2014 Restaurant and Laptop datasets and the ACL Twitter dataset, reporting significant improvement on Restaurant, gains in macro-F1 on Laptop (with accuracy slightly below the strongest baselines), and second-best results on Twitter. This profile is exactly the kind of fine-grained, topic-dependent sentiment that entity-level reputation monitoring requires. I explain further the model and its use in its specific section later in the report.

Topic classification with IPTC NewsCodes

For broad topic classification for the news articles, the pipeline adopts the IPTC NewsCodes taxonomy *(International Press Telecommunications Council, 2024)*, which is the global standard used by Reuters, AFP, and the Associated Press to categorize news content and has 17 predefined categories for news classification (economy, leisure, etc.). Adopting an established taxonomy rather than inventing news categories on my own, has two practical benefits: media professionals already understand the labels, and the outputs slot directly into other news intelligence tools that speak the same vocabulary. The classifier itself, classla/multilingual-IPTC-news-topic-classifier, is validated by *(N°8, Kuzman & Ljubešić, 2025)*. Their model is a fine-tuned XLM-R-large, trained on a corpus of 15,000 news articles in Croatian, Slovenian, Catalan, and Greek that was automatically labelled by GPT-4o (a teacher-student setup that avoided manual annotation). To note that the XLM-R-large backbone handles 100 languages so the model, at inference, is truly multilingual. On a manually annotated multilingual test set the model reaches macro-F1 of 0.746, micro-F1 of 0.734, and accuracy of 0.734, outperforming GPT-4o (gpt-4o-2024-05-13) used directly in a zero-shot setting. They also show that filtering predictions by a confidence threshold of 0.90 raises performance to micro-F1 of 0.798 and macro-F1 of 0.80. That cost-versus-accuracy profile, a small encoder beating a frontier LLM at a fraction of the inference cost, confirmed the model as the production choice for the seventeen-category classification task in Stage 6.

Domain classification with GLiClass

The pipeline also needs a finer-grained tag than IPTC's broad categories: specifically, whether an article relates to tourism subdomains like attractions, lodging, or restaurants. This multi-label domain classification stage is built on GLiClass *(Stepanov et al., 2025)*, which adapts the GLiNER span-matching paradigm to sequence-level classification. The trick is that GLiClass treats labels as natural-language prompts encoded jointly with the input text in a single forward pass, which makes it functionally equivalent to a cross-encoder while being up to 50 times faster than approaches that require one inference pass per label. The authors report that gliclass-large-v3.0 surpasses the strongest cross-encoder baseline (deberta-v3-large-zeroshot-v2.0, F1 of 0.6821) by +0.037 absolute, and that the model maintains strong few-shot performance even with very limited training data. That few-shot behaviour is what made fine-

tuning GLiClass-x-base (the multilingual version) with QLoRA on a small hand-crafted bilingual dataset feasible, instead of having to train a classifier from scratch .

Named entity recognition with GLiNER

The named entity recognition component of the experimental ABSA module is grounded in GLiNER *(Zaratiana et al., 2023)*, the same architectural family GLiClass derives from. GLiNER reframes NER as a span-matching task between text tokens and natural-language entity-type prompts processed through a single bidirectional encoder, which means the user can ask for arbitrary entity types at inference time without retraining. According to the authors, GLiNER outperforms ChatGPT and fine-tuned LLMs in zero-shot evaluations across more than twenty public NER benchmarks while remaining small enough to run on CPU. That last property matters a lot for a pipeline processing high volumes of news articles in a cost-constrained cloud environment, where running a frontier LLM per article would not be financially viable. Being able to customize entity classes beyond the traditional three (ORG, LOC, PER) from other models I tested, made it the easy choice for this pipeline. You can feed it practically any class (political institutions, restaurants, hotels,etc.)

Embeddings, clustering, and retrieval

I tried to move away from LDA learned in text mining, for a better multilingual fit.

The inspiration for applying embeddings to tourism content comes from *(Díaz-Pacheco et al., 2023)*, who propose a comprehensive deep-learning pipeline (CDLM-ITT) for topic discovery and sentiment analysis on news articles about Cancun published in the United States and Canada between July 2021 and July 2022. Their key contribution is showing that transformer-based document embeddings combined with clustering algorithms effectively surface latent thematic structures in tourism news, producing human-interpretable topic outputs supported by GPT-3-generated summary phrases. The fact that their results were readable by domain experts without data-science training validated the decision to use dense multilingual embeddings with cosine similarity and connected-component clustering in Stage 5. Semantic clustering and retrieval both depend on dense multilingual sentence embeddings. The pipeline's embedding backbone is BAAI/bge-m3, chosen for its MTEB leaderboard rankings on retrieval

and clustering tasks and its 8,192-token context window, which is large enough to encode full news articles without truncation.

Late chunking for retrieval-augmented generation

For the experimental retrieval-augmented generation interface, the chunking strategy builds on the late chunking method introduced by *(Günther et al., 2024)*. Conventional chunking splits a document first and embeds each piece in isolation, which means a chunk that says "she announced the expansion" loses the antecedent for "she" the moment it gets cut off from the surrounding paragraph. Günther et al. flip the order: embed all tokens of the long text in a single forward pass first, then apply chunk boundaries before mean pooling. The resulting chunk embeddings carry the full document context. On BEIR retrieval benchmarks measured by nDCG@10, late chunking always beats naive chunking (matching it only on Quora, where documents fit in a single chunk), and the improvement grows with document length. This feature is very helpful for news articles analysis, where pronouns and entity references depend heavily on the surrounding context.

Chapter 1: Methodology and Technical Implementation

This chapter provides a detailed description of the methodology adopted for the pipeline, organized around its nine sequential stages and models used for each stage. Where challenges were encountered that required changes in approach, these are documented explicitly, along with the reasoning behind the pivots.

Overall Architecture: The Nine-Stage Pipeline

The pipeline was designed as a sequential, nine-stage data processing workflow where each stage transforms or enriches a shared data structure (a pandas DataFrame) before passing it to the next stage. The entire pipeline runs as a single containerized job on a daily schedule, the

data volumes are moderate (hundreds of articles per day, not millions), and the linear design is dramatically simpler to debug, monitor, and maintain for a small team. The nine stages are:

Stage	Name	Technology	Purpose
1	Sitemap Scraping	requests, xml.etree, SerpAPI	Discover articles from past 24 hours
2	Filtering/Relevance Analysis	Gemini LLM + Pydantic	Score relevance to QC tourism (0–100)
3	Web Scraping/Browser automation	Playwright, Trafilatura	Extract full article text
4	Sentiment Analysis	twitter-xlm-roberta	Compute sentiment score (–1 to +1)
5	Clustering	BAAI/bge-m3 + cosine sim	Group duplicate/related articles
6	IPTC Classification	multilingual-IPTC classifier	Assign standardized news topic
7	Domain Classification	GLiClass fine tuned	Score Attrait/Hebergement/Restaurant
8	Summarization	Gemini LLM + Pydantic	Summary, perception, geographic tag
9	Data Storage + visualization	Google BigQuery / Looker	Persist to cloud data warehouse and dynamic daily dashboard update

Stage 1: Sitemap Scraping and Article Discovery

The pipeline's first stage discovers all news articles published within the preceding 24 hour window. It parses XML sitemaps and RSS feeds from a curated list of over twenty Canadian media sources, supplemented by programmatic Google News searches via SerpAPI.

The Need to Build a Collection System from Scratch

One of the biggest challenges I faced upfront and early on was that DQc had no existing data to work with, no internal archive, no structured media database, nothing. The original vision was to leverage existing data feeds, APIs (to directly access web content), or aggregated datasets to populate the intelligence pipeline, but in practice, no such resources existed. Quebec's regional media ecosystem does not offer centralized API access, and DQc had no pre-existing data collection infrastructure or historical article databases. This meant that a significant portion of the project's development time, originally allocated for NLP purposes, had to be redirected toward building the entire data collection layer (Stages 1 to 3) from the

ground up. The sitemap parser, the SerpAPI integration, the Google News URL resolver, and the Playwright scraper with browser automation collectively represent a substantial engineering effort that was not originally anticipated in the project scope but proved essential for the system's viability.

Legal and Ethical Framework

DQc and I agreed on a clear boundary: publicly available news content containing no personal information could be collected for internal analytical use, as long as we weren't redistributing it commercially or profiting from the content itself, as it can violate terms and conditions of some websites.

Choosing an Approach

The next question was where to start. I couldn't just download the entire web and analyze everything, that's not feasible in terms of time, cost, or computing resources. I needed a focused, efficient approach. Since I was new to scraping “at this scale”, I spent a lot of time exploring different methods. I watched tutorials, read documentation, and experimented. My first instinct was to build a traditional web crawler, but I quickly realized after testing the code how slow and unreliable that approach was in practice, especially for a production system that needed to run daily. The web crawler really clicks on every single link of a webpage (publicity, uninteresting website sections etc.). Also, due to the fact that the website's servers would be bombarded, we might get rate limited pretty easily.

I also looked into commercial news aggregator APIs like NewsAPI and NewsCatcher that have proprietary web crawlers and much larger servers than my simple code, and I proposed a few of these to the team as an initial data access point to build the NLP pipeline upon it. But due to the monthly enterprise package cost being somewhat out of budget for the company, and the fact that I didn't want the company to commit to paid subscriptions before the NLP pipeline was even built and proven, it felt like putting the cart before the horse.

So I kept looking for alternatives, and that's when I discovered something that changed the direction of the project: nearly every news website publishes XML sitemaps or RSS feeds that one can dig up by accessing either robots.txt or parsing their website structure. These are

machine-readable files that websites create specifically so that search engines (like Google for example) can find and index their content. *(see Figure 2)*. I tried to manually search if the Quebec media outlets I was targeting, offer any kind of API (which grant the most direct access to a website's content) like The Guardian or New York Times, but unfortunately, they don't These sitemaps they offer though, turned out to be a goldmine. Parsing them gave me a respectful, lightweight way to discover new articles every day, since these feeds are explicitly intended for machine consumption.

Sitemap Source Configuration

For the XML part, I built a curated list of over twenty Canadian media sources spanning both languages. I had to research the major relevant news media myself, as the company did not have an internal list (only limited to a few websites). The monitored sitemaps encompass major francophone and anglophone outlets:

Source Domain	Format	Language
La Presse (lapresse.ca)	XML Sitemap	French
Radio-Canada (ici.radio-canada.ca)	XML Sitemap	French
TVA Nouvelles (tvanouvelles.ca)	XML Sitemap	French
Journal de Québec (journaldequebec.com)	XML Sitemap	French
Le Soleil (lesoleil.com)	XML Sitemap	French
Le Devoir (ledevoir.com), 5 sections	RSS Feed	French
La Tribune (latribune.ca)	XML Sitemap	French
24 Heures (24heures.ca)	XML Sitemap	French
Carrefour de Québec	RSS Feed	French
Journal de Montréal	XML Sitemap	French
Global News (globalnews.ca)	XML Sitemap	English
Montreal Gazette (montrealgazette.com)	XML Sitemap	English
National Post (nationalpost.com)	XML Sitemap	English
Globe and Mail (theglobeandmail.com)	RSS Feed	English

Narcity (narcity.com)	XML Sitemap	English
MTL Blog (mtlblog.com)	XML Sitemap	English
Etc …		

Filling the Gaps with Google News and SerpAPI

Going through each website one by one, I discovered that some sites don't have comprehensive readable sitemaps, either poorly formatted, outdated, or not readable by Python libraries (neither the Ultimate site map reader library nor xml.tree), and some with no sitemaps at all, which would require other web crawling techniques that would be too hard to implement. But I wanted the media review to be broad in scope and have a high coverage for the company, so I researched other ways I could add on to the XMLs of the 20 to 25 major websites I already had.

To fill in the gaps for outlets that didn't have well structured sitemaps, I first tried to capture the RSS feed of a news aggregator (that have a legal license to use bots and crawlers at scale for almost all websites) and I was able to discover the Google News RSS available for personal use under their terms. I built my original pipeline by manually parsing the RSS like I did for other websites but had to use the gnewsdecoder Python library to decode the website URLs that Google encrypts in their RSS feed. (you can see in Figure 2 how the http link is encrypted and contains codes and numbers).

I actually hit rate limit errors (error code 503) even though I implemented 7 seconds delays because manually decoding encrypted URLs counted as visiting the page from the same IP address and a Google RSS had hundreds of feeds so too many calls were sent to the same server. I tested extending the delays as it resolved the rate limiting blockage but due to the hundreds of URLs, it would slow down the pipeline significantly, which I didn't want (because Google Cloud charges on the time the pipeline runs ; the longer it takes time for the Cloud Job to finish, the more expensive).Later on, I researched web search tools like Tavily and Brave Search etc. SerpAPI was chosen as it mediated Google News searches , and after investigating how they operate , I found out that they already do what I did manually (parsing Google News RSS among others as well, like Bing News etc.) and returns the scraped URLs for the user. I

realized it was much faster as their servers were much quicker and more efficient than my code.

SerpAPI offered 250 free searches per month, so I implemented six search configurations covering both languages, Canadian and international perspectives, and domain-restricted searches for niche outlets like tourismexpress.com, monvieuxquebec.com, and urbania.ca, etc., since it was not a good idea to parse the whole RSS for those because the percentage of Quebec tourism news was less than 5% for those niche sites, not specifically covering Quebec city news. SerpAPI has built filtering functions (which I discovered how to manage while manually parsing Google RSS myself earlier, as they were using the same techniques I was using manually) to narrow the search and retrieve more niche URLs closely related to Quebec tourism.

Parsing Google RSS manually gave me a good intuition on how to use their filters to access the most news possible, among which using the least possible vocabulary for a query produces the most results because it does not restrict the search. I figured out, after retrieving results, that SerpAPI does not filter Google RSS by 24 hours as well as I did, so I had to implement code to keep only the articles from the past 24 hours so as not to analyze the same articles each day. I did so for all other websites too, as RSS feeds cover more than 24 hours of news and our pipeline runs daily so we need only 24 hours old news articles.

A global budget of eight API calls per run keeps costs under control at $0.

Finally, this hybrid approach (sitemap + SerpAPI) gave me broad coverage without depending on any single source or paid service.

Parsing and Deduplication

Each sitemap URL is fetched concurrently using a ThreadPoolExecutor with five worker threads. The parser handles two XML formats: RSS feeds, identified by the <rss> root element, extracting <link>, <pubDate>, and <title> from each <item>; and standard XML sitemaps, iterating over <url> elements and extracting <loc>, <lastmod> or <news:publication_date>, and <news:title>.

Articles discovered from multiple sources are deduplicated (because it turns out different news outlets do relay “identical, word for word” articles covering the same news) by normalizing URLs, stripping tracking parameters like utm_source, utm_medium, utm_campaign, fbclid, and gclid, and applying exact title matching as a fallback. This dual approach ensures only unique articles proceed to the next stage. This approach helped not to waste resources by analyzing the same articles each day, by filtering from the last 24 hours among the RSS feeds' URLs and making sure we deduplicate same news either by URL or if it has exact same title wording, to avoid analyzing identical (word for word) articles.

HTTP 403 Forbidden Errors and User-Agent Engineering

The first scraping attempts used Python’s requests library with only default headers. Several outlets, including TVA Nouvelles (tvanouvelles.ca), returned HTTP 403 Forbidden errors right away. *(see Figure 3)* After looking into it, I found that their servers were simply checking for standard browser identification headers that any normal web client sends. Adding a proper User-Agent string along with Accept, Accept-Language, and DNT headers fixed the issue on most sitemaps. From there, I built out a small pool of common browser signatures to rotate through, which kept things consistent across the full list of outlets.

The Timezone Parsing Nightmare

Date parsing proved far more challenging than initially anticipated. Each media outlet formats timestamps differently: some use ISO 8601 (2025-11-25T14:28:00+00:00), some use RFC 2822 (Sat, 29 Nov 2025 14:28:00 GMT), some use informal formats (Nov 17, 2025, 12:52 PM), and SerpAPI returns relative dates ("2 hours ago") which I ended up using a regex pattern to filter out 24 hours news. Timezone handling introduced additional complexity as the same date means a different thing in an entirely other time zone. A Bing News item was observed with a "GMT" timezone suffix that Python's standard datetime parser could not handle, requiring a dedicated parsing pathway through email.utils.parsedate_to_datetime(). The final date parser supports over twelve distinct format patterns and three parsing strategies (email-style, ISO, and strptime), all normalizing to UTC time zone before comparison. This parser was developed incrementally over several weeks, with each new source revealing a previously unseen date format that caused articles to be incorrectly excluded from the 24-hour window.

Data Quality and Completeness Issues

Even for freely accessible articles, data quality varied significantly across sources. Some sitemaps contained stale entries. Some articles that appeared in sitemaps returned 404 errors when scraped (having been deleted or moved between publication date and collection time) or the website is under technical maintenance and not accessible. Google News redirect URLs occasionally fail to decode because Google changes their URL encryption from time to time, resulting in lost articles. These quality issues, while individually minor, collectively required extensive defensive coding, null checks, fallback defaults, retry logic with exponential time backoff on retries, and detailed logging, that added considerable complexity to the Python codebase.

Stage 2: Relevance Analysis

URL Relevance Filtering, The Real Challenge

Filtering URLs turned out to be one of the hardest problems in the entire pipeline, and it took several failed strategies before landing on something that actually worked. I had the first intuition that an LLM will be eventually needed for this complex classification/filtering task, but not knowing the volume of data at first, I was afraid the cost induced by LLMs would blow up pretty rapidly. I later tried to test several other methods (although I knew are not 100% perfect) to see if I can, in a way, pre-filter the URL's before feeding them into the LLM.

The difficulty starts at the source level. RSS feeds, which are the primary discovery mechanism I chose, return only URLs, and URLs contain only the article's title, not its content. So, the pipeline is making relevance decisions based on a handful of words in a URL path, with no access to the body of the article. That constraint shaped everything that followed.

The Keyword Trap

Keyword filtering was the first approach tried as a pre-filter, and it failed quickly. Searching for "québec" in URLs returned everything from Hydro-Québec press releases to provincial government announcements, the word is simply too broad to be useful. There's no way to distinguish "Québec City" from "Québec the province" through string matching alone. Adding "tourisme" as a second keyword helped marginally but introduced the opposite problem: plenty of articles deeply relevant to the tourism economy, a new Air Transat route, a hotel labour dispute, a restaurant opening, never use the word "tourisme" at all.

A multi-step zero-shot classifier (facebook/bart-large-mnli model) was also tested as a middle ground: first filter for articles tackling Québec City specifically, then apply a second pass filtering for tourism relevance. This pipeline was meaningfully better than raw keyword matching, but still not reliable enough to trust at scale. Too many genuinely relevant articles were lost in the first filtering step before the tourism classifier even saw them, and the ambiguity problem never fully went away. It became clear that no classification approach working only from titles and URLs, without access to actual article content, was going to be precise enough.

The Case for a Smarter Relevance Model

Another idea was to use embeddings and try to filter out URLs by similarity to “Quebec City tourism”. The idea of using a cross-encoder reranker emerged from a separate context: building a RAG (Retrieval-Augmented Generation) system for internal use. In RAG pipelines, the standard approach is to retrieve candidate documents using cosine similarity on embeddings, comparing a query vector to document vectors and ranking by distance. But cosine similarity has a fundamental weakness: it compares vectors independently, without ever considering the relationship between the query and the document together. Two texts can be close in embedding space because they share vocabulary or domain, not because one actually answers the other.

Cross-encoders solve this differently. Instead of embedding query and document separately and comparing them, a cross-encoder takes both as a single input and computes a relevance score that reflects their interaction directly. It reads them together, which means it can catch

nuances, like a document that uses the right words in the wrong context, or a document that's clearly relevant even though it uses different vocabulary than the query. For ranking tasks, it is widely known that cross-encoders consistently outperform cosine similarity in precision. That's why I gravitated towards Cross-encoders rather than simple cosine similarity.

The idea was to apply this to the URL filtering problem: use a reranker to score each article title against queries like "Québec City tourism" and filter out low-scoring ones before passing the remainder to the LLM (which I finally ended up using). This would reduce the volume hitting the LLM and hopefully catch the most obvious irrelevant content cheaply, because the caveat with LLM as discussed earlier, is that they are powerful but can be very costly depending on the volume of data input/output. (I had no idea of the size of data the pipeline, fully scale-up, was going to daily analyze at first.)

Why Reranking Didn't Save the Day

Several reranking models were tested in the hope that a fast, lightweight scoring step could pre-filter articles before they reached the LLM. The results were promising for filtering noise but discouraging across the board, though for different reasons.

Jina Reranker v3 was the most promising. It produced directionally correct scores and handled French text well thanks to its multilingual training. The issue was that its raw outputs are logits with no built-in normalization, so the numbers themselves are meaningless without post-processing *(see Figure 4)*. Trying to put a threshold number as to what is noise or not was hard. Temperature-scaled sigmoid helped make the scores more readable, *(see Figure 5)* but choosing the right temperature was pure guesswork, and even with well-distributed scores, drawing a hard classification threshold remained arbitrary. An article scoring 0.4 might be genuinely irrelevant, or it might be a highly relevant article written in a way that doesn't surface obvious tourism signals. There was no principled way to tell.

Zerank-1-Small crashed on GPU memory. Even though I tried 4 bit quantization to reduce model memory usage. This is a structural limitation of cross-encoders more broadly: because they process the query and the full document together in a single forward pass rather than

encoding them separately, memory consumption scales with the combined input length. Especially that model was already big and consumes high RAM even before inference.

BGE-Reranker-v2-Gemma, which uses a 2 Billion parameter Gemma base that is an open source LLM free to use locally without API, fared better on short inputs because it supported adding an explicit instruction alongside the query (being an LLM), which meaningfully improved results by giving the model clearer context about what "relevant" actually meant in this setting. However, extending it to full-article multi-category classification would have introduced the same threshold and calibration problems as the other rerankers.

Voyage AI's rerank-2.5 compressed all scores into a narrow band regardless of how the queries were phrased. When every article gets a score between 0.23 and 0.49, there's no threshold that separates relevant from irrelevant and I couldn’t figure out a way to normalize these scores in a meaningful way.

The fundamental issue underlying all of these failures I guess was architectural. A score of 0.7 from a reranker means "more relevant than the other documents in this batch," not "this document is 70% likely to be about tourism." Repurposing that signal as a binary classifier requires picking a threshold that the model was never trained to respect, which is why every attempt produced either too many false positives or too many false negatives. Although the obvious bad URL’s were pretty well filtered out (+/- 80% of URL’s on average), *(see Figure 14), (note: Figure 12,13 and 14 gives an overview of how the reranker (second excel column with green bars) compared to other approaches I tested namely the LLM (blue bars) and Agentic LLM (pink bars) that I will explain after this section)*.

I did compare the Reranker results though eventually to the LLM (final method used) , it was great at filtering the noise (irrelevant articles) and seemed promising, the issue is with no clear threshold we would lose good articles as well, and recall of articles is crucial for DQc (for them not to miss important news) so I couldn’t afford to lose good articles even if we clear out 80% of the irrelevant articles before feeding the LLM. I moved away from rerankers and kept it for RAG purposes only (in another stage).

Letting the LLM Handle it, with a Web Search tool attached to it

The truth is that no matter how capable the underlying model, working from a URL and title alone gives any classifier too little to go on and be good. The title of a news article is often vague, abbreviated, or written for click appeal rather than descriptive accuracy , and for a niche geographic and thematic target like Québec City tourism, even a well-written title frequently omits the specific signals that would make relevance obvious.

Large language models offered a genuine improvement here, and not just because they're larger. A large enough LLM, trained on the breadth of the pretty much the whole web, carries implicit knowledge about places, institutions, companies, and events , so even when a title doesn't explicitly mention tourism, the model can recognize that "Mont-Sainte-Anne" is a ski resort in the Québec City region, or that "Air Transat" operates routes relevant to the local tourism economy. That kind of entity level world knowledge is something no reranker or zero-shot classifier can replicate. Gemini was the natural choice given that DQc operates within Google Cloud, where it integrates cleanly into the existing infrastructure.

Choosing the right LLM :

Each LLM has attributes: knowledge cutoff and benchmarks on reasoning as well as input/output token cost, latency etc.. I referred to a very useful website: https://www.vellum.ai/llm-leaderboard, which gives the latest LLM leaderboard and their performance on several tasks. For us, reasoning was key although URL filtering is not considered PhD level reasoning, but the reasoning benchmark still gives good indication on the model capability . I test Gemini 2 and 2.5 but got fewer relevant URL’s each time, Gemini 3 Flash returned the most relevant URL’s (which I had no choice but to inspect manually) and results were very close with Gemini 3 pro. Gemini 3 flash is a big bump in reasoning (90% on the DIAMOND test) from 2.5 (around 78%) , but going to pro did not justify the 5 times increase in price. So Gemini 3 flash was the sweet spot ; prompted as a media intelligence analyst with detailed instructions about Québec City's geographic scope, what counts as tourism-relevant, and what doesn't. This approach could resolve the ambiguity that keyword matching and reranking couldn't, because it reads context rather than matching strings.

Agentic AI final solution

Even so, the LLM had its own failure mode. When a title was genuinely ambiguous or unrepresentative of the article's actual content, the model had no real basis for a decision and would sometimes assume a hallucinated relevance judgment rather than admit uncertainty. The obvious fix?: scraping the full article content before scoring, which was ruled out early. With relevant articles representing only around 5% of the total volume, fetching the full text of every URL would be prohibitively expensive and slow, most of it wasted on content that was never going to be relevant anyway and will blow up LLM costs.

The final solution was to make the process “agentic” by attaching to the LLM, a web search tool I discovered later on, and was available on the Cloud. “Google web search custom JSON API” which was a paid service available, but very affordable. While not fully a scraper, it had still access to the title and a short snippet of the articles (first 200 words more or less). So, rather than forcing the LLM to commit to a score on every article, it was given the ability to recognize its own uncertainty and act on it, triggering a targeted web search when confidence was low, retrieving the snippet of the actual article content, and revising its judgment with that additional context. I put 0.8 out of 1 as a confidence threshold to trigger a search. This preserved the efficiency of title-only scoring for the clear cases while giving the model a way to resolve the ambiguous ones without scraping everything upfront.

I ran multiple tests with this agentic framework , the LLM tended to do a search on 30% of all URL’s (when LLM confidence was below 80%) *(see figure 11)* which is significantly more efficient than analyzing all URL’s and help reduce the cost of the API (approx. 7$ per 1000 requests) by 70% as well as input tokens fed to the LLM and the cost of output tokens the LLM used for providing its reasoning on the relevance score. This method gave the cleanest results and was the closest I could find to filtering URL’s without scraping and downloading the whole web and feeding it to an LLM which would be unfeasible and enormously costly in time and money. (*(see Figure 12, 13, 14 for comparison between LLM (in blue) , Reranker (in green)* agentic LLM results which are in pink , first pink column is relevance score, second pink column is the LLM’s confidence score)

System Prompt Design ;

The agentic framework in place, I tried to test different prompts and check the LLM adherence. The relevance analysis relies on a meticulously crafted system prompt of approximately 1200 tokens (purposely big to be able to be cached on the Cloud) that positions Gemini as an "elite Media Intelligence Analyst." The prompt encodes five components:

1. **Geographic Scope Definitions:** Precise enumeration of five geographic zones (the Quebec City agglomeration and the MRCs of La Jacques-Cartier, La Côte-de-Beaupré, L'Île-d'Orléans, and Portneuf) with specific municipalities and tourism assets. This was intended for relevance scoring about Quebec City.
2. **Classification Rubric:** Four-tier scoring, Highly Relevant (90–100) for news directly about Quebec City tourism, Moderately Relevant (80–89) for indirect news (for example provincial news) about politics, economy or non "purely" tourisitc news in the strict sense but might affect Quebec City tourism in an indirect way. Then, Low Relevance (10–79) namely Canadian news with no impact, Not Relevant (0–9) , mainly international news.
3. **Contextual Analysis Instructions:** few shot prompting achieved better prompt adherence than zero shot as LLM considered very local trivial events like the weather or a local incident as impactful on Quebec City tourism but DQc was not interested in those articles. Few shot prompting gave the LLM more nuance.
4. **Key Indicator Extraction:** Requirements to identify 1–3 named entities justifying the classification and justify reasoning (telling an LLM to output its reasoning and not just a relevance score without explanation, generally improve accuracy)
5. **Output Format:** Strict JSON schema enforced through Pydantic validation for structured output (the use of Pydantic will be explained later in the report)
6.

The LLM Quota Crisis

At first the code ran smoothly but problems with API rate limiting from Google Cloud started to appear with further testing.

In fact, the first implementation of Gemini relevance analysis processed articles individually, one API call per article. On a typical day with 1500 discovered articles, this meant 1500 Gemini API calls for a single pipeline run. The Google Cloud Platform's per-minute quota (100 API calls per minute) for the Gemini model although not exceeded, I still got 429 (Quota Exceeded) errors *(see Figure 7)* that caused roughly 30–40% of articles to fail with zero scores and ERROR categories. The pipeline logs showed the devastating pattern: successful scoring of an article, then a 429 error, then another success, then two more 429 errors, creating an unpredictable patchwork of scored and unscored articles. It took me a long while to discover what was the issue, it turned out that Google Cloud has a "shared" pool for all users using LLM's and due to my several tests, it was exceeded for a specific day/week/month.

The solution was batch processing grouping 25 articles into a single API call and asking Gemini to score all of them at once. *(see Figure 10 for batch of 5 early implementation).* This reduced the number of API calls from 1500 to 60 per run, an efficiency improvement of 96%. However, batching introduced new complexities: accuracy of the LLM dropped, and it turned out to be an indexing issue. I clearly indexed the article in each batch so not to confuse the LLM and so that every relevant analysis corresponds to the article in the same order. The number 25 was chosen as the optimal balance between API efficiency (fewer calls) and response quality (the LLM's accuracy degraded with extremely large batches, and I got fewer relevant URLs filtered). The resource exhausted 429 errors, disappeared for a few weeks but returned here and there still. After a long while of not knowing what to do still, I tried putting a delay of 2 seconds between each API call because I thought it might be a burst rate issue. The errors were completely absent after that and LLM calls ran smoothly.

Batching helped dramatically reduce the input token cost caused by the prompt being invoiced for each article. This way one prompt is sent for 25 article URL's. I did cache the prompt as well as a cached input has a different cost rate on the Cloud and it helped reduce the input cost by 90%. It is a feature available on the Cloud that helps put the prompt into a cached memory

storage that speeds up same prompt calls at a large amount. I made sure to delete the cache after the LLM calls are done because Google Cloud charges for cache storage per minute usage.

Output Errors

Despite setting Gemini's temperature to 0.0 for deterministic output and maximal prompt adherence, occasional other variations in output format, JSON structure, and classification decisions output errors were observed (*see Figure 8, the exclamation mark errors I observed with both Gemini and GPT models I tested early on*). I guess all these issues stem from the inherent unreliability of working with a probabilistic large language model. Even when prompted to output a specific format that can be fed into a dashboard, it deviated and would not adhere to prompt from time to time causing articles to be skipped as the output format was not proper JSON. Also output tended to blend English and French causing JSON parsing errors as well. I guess because the articles were multilingual, the LLM did not stick to a single language output.

Even though the Cloud provided JSON response Schemas that you can implement when invoking an LLM, errors were still happening such as the model returning "neutral" instead of "neutre" or wrapping JSON in markdown code fences. I had to use “Pydantic” which is a Python library (which I had to dive deep into and discover) used as a validation layer with field validators, manual JSON parsing fallback, and multi-attempt retry logic for French/English mix-up , that were all implemented to address this fundamental unreliability and help with what is called “structured output”.

These errors were more significant in the LLM summary step and perception tagging, that will be explained later on, where I had to use more in depth Pydantic functions as well for output stabilization.

With all these implementations, output format errors dropped from approximately 5% of batches (for Gemini) to almost zero. By the very end of development, I had no remaining issues and the pipeline ran smoothly with no errors and there were no more “null” values in the Looker dashboard resulting from skipped articles due to malformed JSON output or wrong entries. I still kept retry logic in code (*see Figure 9),* for extremely rare errors due to GCP rare servers unavailability.

Stage 3: Web Scraping & Browser automation

After articles pass through the Gemini relevance filter (Stage 2, scoring above 79.5 to keep news impacting Quebec tourism both directly and indirectly), the pipeline moves to extracting the full article text from each web page. This stage went through several iterations before I arrived at a reliable solution.

The thing is each website has a different structure. What I used to do when scraping was parse the HTML page of a website and using a regex pattern to filter out relevant sections (mainly paragraphs sections etc.). But this approach would not be feasible as we cannot explore with simple code all kinds of website structure. On top, some websites have dynamic content and use JavaScript on top of HTML and many other variations. I had to search for advanced libraries with active communities behind them that were far more professional at website parsing and scraping than my simple personal code. Luckily for news articles, there were such libraries. I chose Trafilatura as it is the best in class in terms of scraping accuracy. Although the library was found, other limitations surfaced :

My first approach bypassed browser automation entirely, using Trafilatura's built in download capability to fetch and extract article content in a single step. This worked for some outlets but produced catastrophic results on others. TVA Nouvelles returned connection errors, and Narcity returned login page HTML instead of article content ,the scraper captured text like "Need an account? Login. Already have an account? Free. $0. Gain access to free articles..." instead of the actual news article. These failures demonstrated that many modern news websites require JavaScript execution to render their actual content, making simple HTTP-based scraping fundamentally insufficient.

My first attempt at browser automation used Selenium WebDriver with ChromeDriver, a widely-used tool for browser automation. While Selenium successfully rendered JavaScript-heavy pages, it presented significant operational challenges: ChromeDriver version management required exact version matching between ChromeDriver and the installed Chrome browser, it had poor async support making concurrent scraping difficult since Selenium is fundamentally synchronous. After extensive testing, I decided to migrate to Playwright.

Playwright, a browser automation library maintained by Microsoft, offered everything Selenium couldn't: native async support essential for concurrent scraping (on distinct webpage domains so as not to bombard the same website at once), built in browser management with no version matching required, and a more modern API. Unlike simple HTTP requests, Playwright opens a full Chromium browser, executes all the JavaScript, and renders the page exactly as a real user would see it.

Resolution occurred in two phases: failed URLs fall back to Playwright-based resolution, opening the News page and waiting for the JavaScript redirect to complete.

Each request creates a fresh browser context with a randomized User-Agent from five Chrome, Edge, and Firefox signatures, randomized viewport sizes (1440x900 or 1920x1080), locale set to fr-CA for French domains and en-US otherwise, and matching sec-ch-ua headers. Resource blocking is applied to images and media to speed up page loads.

One particularly insidious problem emerged only after the pipeline had been running for several days. Reviewing the scraped content in the BigQuery table, I discovered that several articles contained cookie consent banner text mixed into the article content. For example, one article from Ville de Québec website had its content prepended with "Ce site utilise des témoins de navigation (cookies) essentiels pour son bon fonctionnement..." *(see Figure 16)* This happened because the cookie consent overlay was part of the page DOM when Trafilatura extracted the content. The solution was to implement an automated cookie consent handler that detects and clicks accept/agree buttons before content extraction, using role-based button detection with regex pattern matching in both English and French, with CSS selector fallbacks for common consent banner implementations. This error did not go away even with the consent handler so I also added extra time for page loading , to give time for Cookies consent to be accepted before moving onto article content and scraping it, as it turned out scraper was too fast as scraping the page as soon as any content loads.

Once the page was properly rendered and cleared of overlays, the HTML is passed to Trafilatura for content extraction, configured to include tables but exclude comments. Trafilatura is a library specifically designed for isolating main article text from navigation, advertising, sidebars, and boilerplate. Articles yielding more than 100 characters of text are

marked as successfully scraped to eliminate page not found 404 errors when a webpage is recently discontinued.

All these errors seem harmless or trivial but still crucial to fix as the Dashboard will look not professional and contain a lot of noise so I had no choice.

Concurrency is controlled by an asyncio Semaphore with a limit of five simultaneous scraping tasks. Per-domain rate limiting ensures 6.8 to 13.6 seconds between requests to the same domain to not bombard websites. A blocklist of website domains that prohibit scraping after some usage was implemented so as not to try to scrape these websites unnecessarily .

Several of the most editorially important media outlets in Quebec operate behind soft or hard paywalls. Le Devoir limits free access to a small number of articles per month, while La Presse uses a metered access model. *(see Figure 17)* The pipeline captures whatever content is available in the freely rendered HTML, but paywalled paragraphs that require authentication remain inaccessible.

That being said, a deliberate decision was made not to attempt to circumvent paywalls. While techniques exist for bypassing metered access, cached versions, archive.org lookups, deploying such techniques in a production system operated by a public-sector organization would raise serious legal and reputational risks. The pipeline's design reflects a principled commitment to accessing only freely available content, even at the cost of incomplete coverage. This ethical boundary was discussed and endorsed by DQc's management team, who agreed that the reputational risk of paywall circumvention far outweighed the intelligence gain. That being said, with the light and well-crafted scraping strategy I implemented, we rarely exceeded the free-tier access to the news on these websites.

Stage 4: Sentiment Analysis

(The data engineering side of the project being finally over, I could move on to NLP)

Assessing the sentiment of a full news article is harder than it sounds. A single article can contain positive facts, negative quotes, neutral background information, and mixed opinions all in the same piece. A story about a new hotel opening might praise the architecture but

criticize the impact on local traffic, so how do you assign a single sentiment to that? This was a question I had to think carefully about before choosing an approach that can fit into DQc dashboard as a single word or number for a sentiment score.

From Bag-of-Words to Deep Learning

My first instinct was to use a vocabulary-based approach. The idea is simple: build a dictionary of words where each word has a pre-assigned sentiment score, then count up the positive and negative words in a text to get an overall score. This is the logic behind well known tools like VADER (Valence Aware Dictionary and sEntiment Reasoner) and the AFINN lexicon. I experimented with this approach early on, and while it was easy to understand, it had serious limitations. It couldn't handle negation well ("not bad" would score as negative because of "bad"), it missed context entirely, and it struggled with French text since most sentiment dictionaries are English-focused. For a bilingual pipeline processing hundreds of articles daily, this approach was too fragile and too slow to adapt.

I briefly considered a document-term matrix (DTM) approach (as it was faster than word counting), where you represent articles as numerical vectors based on word frequencies and then train a classifier on top. This is a step up from raw dictionary lookup, but it still treats words as independent units and loses all contextual meaning. A phrase like "the hotel was not disappointing" would be treated the same as "the hotel was disappointing" because the same words appear.

That's what pushed me toward deep learning and transformer-based models. These models understand context, handle multiple languages natively, and have been pretrained on massive amounts of text. The question was: which model?

Testing Models Without Data

At the start of the project, I had no scraped data yet, or very few, the pipeline hadn't been set to automatic daily run yet. So I needed an external dataset to benchmark different models. I found the “mx-phd/tourism” sentiment dataset on Hugging Face, a multilingual (French, English, and Spanish) dataset of tourism-related tweets manually annotated at the text level with sentiment labels and at the token level with locations and thematic concepts linked to

the World Tourism Organization Thesaurus of Tourism and Leisure Activities, covering the Pyrenees/Basque Country cross-border region between France and Spain, with 1,662 training samples, 619 validation samples, and 680 test samples. It wasn't an extremely perfect match for news text, but it gave me a starting point to compare models in a tourism context. I tested six multilingual sentiment models .

1. cardiffnlp/twitter-xlm-roberta-base-sentiment, based on XLM-RoBERTa, trained on multilingual tweets (Barbieri et al., 2022; Conneau et al., 2020)
2. nlptown/bert-base-multilingual-uncased-sentiment, BERT-based, trained on product reviews with a 1-to-5 star scale
3. tabularisai/multilingual-sentiment-analysis, a multilingual model optimized for general sentiment
4. cardiffnlp/twitter-xlm-roberta-base-sentiment-multilingual, an extended multilingual variant of the CardiffNLP model
5. clapAI/roberta-base-multilingual-sentiment, RoBERTa-based multilingual sentiment classifier
6. joeddav/xlm-roberta-large-xnli, a large XLM-RoBERTa model used in zero-shot classification mode

I had 95.1% accuracy off the bat with Twitter-XLM-Roberta *(see Figure 18)* which was a good sign. I needed to test to see if tourism datasets are generic or too niche like the finance domain for example where you need more specialized models (like finBERT for example trained on finance dataset), it turns out tourism is mostly common and not too niche and sophisticated data.

Stress-Testing with Synthetic Data

The tourism dataset was useful for the first pass, but it contained mostly straightforward, easy-to-classify examples. I wanted to know how these models would handle harder cases ,sarcasm, mixed sentiment, negation, and implicit opinions , which are common in real world social media news and editorial writing. Since no such dataset existed for bilingual tourism

content, I used an LLM to generate a synthetic challenge set of 100 carefully crafted harder examples in both English and French. Each sample was tagged with a difficulty level: medium,hard, sarcasm, mixed sentiment, tricky negation, false positive, and so on.

The results on this challenge set were revealing. Performance dropped significantly across all models compared to the standard tourism dataset. *(see Figure 19)*

The degradation in performance was expected; the synthetic set was deliberately hard. Sarcasm accuracy was especially poor across the board. French sarcastic examples like "Formidable expérience si vous aimez les cafards et les serviettes humides. Cinq étoiles pour les cafards" were confidently classified as positive, which showed a clear limitation of all models tested. This confirmed that no off-the-shelf model would perfectly handle every edge case, and that the choice of model had to be made with the actual use case in mind.

The Decision: Fine-Tune or Not?

The question of whether to fine-tune a model on domain specific data could only be answered once I had real data flowing through the pipeline. I waited until I had collected roughly a month's worth of scraped news articles and then manually reviewed a sample of 185 articles to assess the tone and style of the content the pipeline was actually processing.

The key finding was that Quebec news articles are overwhelmingly formal in register. Unlike tweets, reviews, or social media posts , which are casual, emotional, and full of slang and sarcasm, news articles use structured, professional language with clear attribution and factual phrasing. This formality actually works in favor of pretrained sentiment models, because the language is less ambiguous and the sentiment signals are more explicit.

When I evaluated cardiffnlp/twitter-xlm-roberta-base-sentiment against this real-world news sample nearing the end of the project time, performance was strong: 88.2% accuracy with a macro F1 of 0.878, close enough to the mx-phd/tourism dataset. The model handled formal news language well, correctly identifying sentiment in straightforward reporting and editorial content. The main confusion areas were between neutral and mildly positive articles, which is a genuinely ambiguous boundary even for human annotators.

Based on these results, I decided not to fine-tune. As a side note, I actually left the fine-tuning decision for all models, to the very end of the project, because I wanted an up and running pipeline the company can at least use as the primary goal. The pretrained model performed well enough on formal news text that the engineering effort and ongoing maintenance burden of fine-tuning would not be justified. Fine-tuning would also introduce a risk of overfitting to the specific articles available at that time, potentially reducing generalization as the media landscape evolved.

Final Implementation

To handle the nuance problem , where a full article contains mixed signals , each article is decomposed into sentence level phrases (minimum 20 characters, maximum 350, split on sentence-ending punctuation using a regex pattern). Each phrase is classified individually in batches of 16 for speed, and the article level sentiment score is computed as:

Sentiment = (Positive – Negative) / (Positive + Negative)

This yields a score in the range [–1, +1], where neutral phrases (with sentiment score = 0) are excluded from the denominator to focus on opinionated content. This sentence level decomposition approach means that a mixed article naturally produces a score near zero, while a strongly positive or negative article pushes toward the extremes.

I used this approach to not deal with LLM that usually gives a single, non-nuanced, sentiment value to an article and cost more than free pretrained model. Although this approach was effective at classifying generally bad news from good news, it was still an objective sentiment. DQc supervisor noticed some articles were bad in tone but not necessarily bad as from DQc or Quebec City point of view. An example was an article about their CEO resigning, while the news was mostly negative in tone which xlm Roberta captured (for example: a sad day that CEO was resigning etc…) , the news did not portray the CEO as a bad person but rather about the sad news he is leaving. So, I eventually added (in the later summarization step , to avoid increasing input tokens and API calls) a “subjective” sentiment prompt to Gemini to analyze subjective sentiment, from the point of view how the image of Quebec city and its entities are portrayed in article. Training a BERT model for that seemed too hard as it requires a large amount of training data on Quebec City.

Stage 5: Semantic Article Clustering

Once all articles have been analyzed for sentiment, the next step is figuring out which articles are talking about the same story (even though not word for word but still mostly same semantic information). If five different outlets all publish articles about a new hotel opening in Old Quebec, I want those grouped together so DQc can see the full picture of how a single story is being covered across the media landscape rather than looking at five separate entries that are really about the same thing.

Even though we did deduplicate earlier by same URL and same title, clustering / grouping was still necessary if different news outlets (so different URLs) relay the same story (although not identically) where the title doesn't necessarily match word for word, but the content was "almost" similar. So semantic analysis was necessary for better dashboard readability and visualization.

Choosing the Embedding Model

To group articles by similarity, I first needed a way to represent each article as a numerical vector, essentially converting text into a point in a high-dimensional space where similar articles end up close together. This is what sentence embedding models do.

Choosing the right embedding model started with the MTEB (Massive Text Embedding Benchmark) leaderboard available on HuggingFace (link : https://huggingface.co/spaces/mteb/leaderboard), which ranks embedding models across a range of tasks like retrieval, clustering, classification, and semantic similarity. It's a useful starting point, but it's important to keep in mind that the MTEB leaderboard gives a general idea of performance, but it doesn't tell you how a model will perform on your specific data, especially in a bilingual French-English news context. So, I treated it as a shortlist, not as a final answer. The best models are usually tested on a validation dataset.

Testing some models

From the leaderboard, I narrowed down to a few candidates. I initially tried smaller, faster models like the paraphrase-multilingual family, which are popular and lightweight and

completely free. But I quickly ran into a problem: after I explored these models via a Python code to explore the Transformer architecture, they turned out to have a maximum context window or token limit of only 128 or 256 tokens, which is far too short for news articles. A typical Quebec news article runs anywhere from 500 to 2,000 words. With a small context window, the model would only encode the first few sentences and ignore the rest of the article, which meant two articles about completely different topics could end up with similar embeddings just because they happened to start with similar introductory paragraphs. The clustering results were poor and unreliable.

Trying to find the right balance between model size and non API costly models like gemini 001 embeddings family, I went towards BAAI/bge-m3, which is a battle tested widely used embedding model free to download and can run on a 16GB CPU (which I ended up using in the Cloud deployment) , quite fast and has a context window of 8,192 tokens , large enough to encompass the full text of virtually every article in the pipeline. It also handles multilingual content natively, produces 1,024-dimensional embeddings, and ranked highly on the MTEB leaderboard for both retrieval and clustering tasks. I validated the model's performance manually over the course of about two months, regularly reviewing the clusters it produced to make sure articles that should be grouped together actually were, and that unrelated articles weren't being lumped in by mistake. Over time, I became confident that bge-m3 was producing meaningful, reliable groupings for this specific use case.

Why Graph-Based Clustering Instead of a Traditional Algorithm

Once every article is encoded into a vector, the natural next step is to cluster them. The typical approach would be to use a clustering algorithm like K-means, DBSCAN, or hierarchical agglomerative clustering. I considered these but decided against them for practical reasons.

K-means requires you to specify the number of clusters in advance, which is impossible in this context. I have no idea how many distinct news stories will emerge on any given day. It could be twelve, it could be forty. DBSCAN handles this better since it discovers clusters automatically, but it relies on density parameters (epsilon and minimum samples) that are

sensitive to the data distribution. These would need constant tuning as the volume and diversity of articles changed over time, and they are notoriously hard to optimize.

Hierarchical clustering with HDBSCAN, which I used earlier when experimenting with BERTopic (inspired by the ResearchGate article from Díaz-Pacheco, 2023), has too many hyperparameters that proved hard to optimize toward a high coherence score. The data did not help either: there were not many data points, and the majority were unique articles rather than duplicates. As a result, most articles ended up flagged as outliers, which either pulled the coherence score down or made it unrepresentative. The data sparsity in the early stages of the project made optimization even harder. I tried Optuna with both random search and Bayesian search, but I could not push coherence scores high enough with so few data points.

That said, this turned out not to be a loss, because the goal here was to assess similarity, not to group by theme. Even if two articles cover the same event but one outlet takes a surface-level angle while another goes deep, I do not want them clustered together. No clustering algorithm offers that level of semantic precision out of the box without significant tuning.

Instead, I went with a simpler and more transparent approach: threshold-based graph clustering. The logic is straightforward. After encoding all articles into vectors, I compute a pairwise cosine similarity matrix, essentially measuring how similar every article is to every other article. Any pair with a cosine similarity above 0.85 is linked as an edge in an undirected graph. Then I extract connected components from that graph using depth-first search. Each connected component becomes a cluster: a group of articles that are all sufficiently similar to at least one other article in the group.

The beauty of this approach is that it requires only one parameter: the similarity threshold. That single number (0.85 for the similarity threshold) is easy to understand, easy to validate by reviewing actual article pairs, and easy to adjust if needed. It also handles a variable number of clusters naturally. If there are three big stories and twenty standalone articles on a given day, the algorithm produces three clusters and leaves the rest unclustered. Articles that do not match anything else retain a cluster ID of zero, meaning they are standalone stories. In practice, around 80% of articles ended up in Cluster 0.

Stage 6: IPTC Topic Classification

I thought about another classification for better dashboard readability .Since every news article covers a topic , politics, economy, sports, arts, science, and so on and the media review was deliberately made to cover a large scope of news either related directly to tourism or having an indirect impact on the tourism landscape (e.g. : politics, economics etc.). To make the scraped data useful for DQc's dashboard, each article needs to be tagged with a standardized topic category so the team can filter and analyze coverage by subject area. For example, they might want to see all articles related to "arts, culture and entertainment" or track how much coverage "economy, business and finance" is getting relative to "lifestyle and leisure."

I chose to use the IPTC (International Press Telecommunications Council) taxonomy for this purpose. IPTC NewsCodes are the global standard used by news agencies like Reuters, AFP, and the Associated Press to categorize their content. Using this established standard rather than inventing custom categories means the classification is consistent, widely understood, and compatible with how the international news industry already organizes content.

The model I used is classla/multilingual-IPTC-news-topic-classifier, which assigns each article one of 17 standardized IPTC top-level categories including society, politics, economy/business/finance, sports, arts/culture/entertainment, science and technology, health, education, environment, crime/law/justice, disaster and accident, weather, religion, labour, human interest, lifestyle and leisure, and conflict/war/peace. The model was trained on a large corpus (like AG news, etc.) of professionally categorized news articles from multiple languages, which makes it a natural fit for this pipeline since it was specifically designed for the exact task I needed, classifying real news content, not social media posts or product reviews. It handles both French and English natively, which is critical for the bilingual Quebec media landscape. The model performance and capabilities were already discussed in the literature review.

The input text for classification is constructed by concatenating the article title with the first 500 characters of content. The model has a maximum token limit of 512, but in practice this is not a limitation. News articles follow a well established journalistic convention called the

inverted pyramid structure, where the most important information, the who, what, where, when, and why , appears in the headline and the opening paragraph. By the time you've read the title and the first few sentences, you almost always know what topic the article covers. In my experience reviewing hundreds of classified articles over several months, the topic is identifiable from the first 500 characters in well over 99% of cases. Feeding the entire article would not meaningfully improve classification accuracy but would significantly slow down processing.

Articles are processed in batches of 8 for efficiency. The model outputs an IPTC category label in English, and a hardcoded French translation map provides bilingual labels ,stored as iptc_topic and iptc_topic_fr , so that the downstream Looker Studio dashboard can display topics in whichever language the user prefers.

Stage 7: GLiClass Multi-Label Classification

What I was Trying to Do

I had a specific problem: given a piece of text (a news article, a listing, a description, in either English or French), we needed to automatically tag it as being about one or more of three categories: attraction (tourism events, museums, destinations), lodging (hotels, B&Bs, accommodations), or restaurant (dining, food establishments), which are the categories DQc told me they would most want to filter and monitor as relevant info to update their website and advise their shareholders. Some texts belong to none of these categories, and some belong to more than one. So this was a multi-label classification task that needed to work in both English and French and handle genuinely difficult cases like the French word "restauration" (which means renovation, not restaurant in some cases), or "Hôtel de Ville" (which means City Hall, not a hotel), or "chambre de commerce" (chamber of commerce, not a hotel room).

Starting with Zero-Shot on Synthetic Data

Since I didn't have scraped articles yet at the beginning, I needed a way to test the model before real data was available. I started by testing the off-the-shelf GLiClass-x-base model (built on DeBERTa v2) in zero-shot mode on an LLM generated synthetic bilingual dataset of 100 EN+FR samples I created to represent the three categories. The results were mixed. Attraction only reached 71.1% accuracy, lodging 79.5%, and restaurant performed best at 94.0%. *(see Figure 20)* The model was picking up on restaurant-related language fairly well but was clearly struggling with broader or more ambiguous categories like attraction.

Moving to Real Scraped Data

Once the scraping pipeline was running, I used a long context LLM to extract paragraphs from a month's worth of scraped articles that pertained to the three categories. I then re-tested the zero-shot GLiClass model on this real DQc tourism data *(see Figure 21)*. Performance shifted: attraction hit 77.7%, lodging jumped to 89.3%, and restaurant held steady at 94.4%. While lodging and restaurant were looking more reasonable, attraction was still lagging and producing too many false negatives, meaning real attraction-related articles were being missed.

The Label Wording Problem

At this point I realized the model's performance was heavily dependent on how the labels themselves were worded. "Attraction" as a single label was too broad and vague. The model was failing to tag articles about museums, or resorts as attractions. I experimented with expanding the label to something more descriptive like "attraction museum touristic event resort" to give the model more signal. As shown in the results, this did push attraction accuracy up to 86.1% and restaurant to 94.7%, but it came at a cost: lodging dropped to 77.7%, likely because the longer, noisier label was creating interference across categories. *(see Figure 22)* It became clear that zero-shot classification with multi-word label engineering was a fragile approach. Performance was too sensitive to label phrasing, and there was no clean way to improve one category without hurting another.

The Dataset I Built for Fine-Tuning

Rather than keep fighting with label wording, I decided to fine-tune the model properly on real labeled data, using clean single-word labels. I used the scraped dataset I gathered from a month of the pipeline's scrapings consisting of 642 labeled examples in both English and French generated by a long context LLM . I made sure its only paragraphs no longer than 512 tokens to construct test set to not exceed model limit. This wasn't a random dataset; every example was written deliberately to teach the model what we cared about. The dataset included easy examples for each category like straightforward sentences such as "Montmorency Falls welcomes over a million visitors each year" (attraction) or "Le Fairmont Château Frontenac a complété une rénovation de 75 millions" (lodging). It also included tricky hard examples designed to catch common mistakes, such as "La restauration des fresques de la chapelle a nécessité 2 millions de dollars" which is about renovation and should NOT be tagged as restaurant, or "Hôtel de Ville de Québec will open its doors to the public for Heritage Day" which is City Hall and should be tagged as lodging. I also included "none" examples covering texts about politics, sports, crime, weather, business, and science ,things a tourism classifier should ignore entirely , as well as multi-label examples for texts that legitimately belong to more than one category, like a hotel that also has a famous restaurant. Each example was tagged with its language and difficulty level so I could track exactly where the model was struggling.

The Architecture: What I Actually Fine-Tuned

First, I extracted the DeBERTa encoder from GLiClass. GLiClass is a wrapper around a DeBERTa v2 model of about 280 million parameters, *(see Figure 23)* and pulled out just the encoder , the part that reads text and turns it into numerical representations. Then I quantized it to 4-bit precision using QLoRA's NF4 quantization, which shrinks the model's memory footprint by roughly 8x while keeping most of its intelligence. Instead of changing the original model weights directly, I attached small trainable LoRA adapter layers to the attention mechanism, specifically to the query, key, value, and dense projections inside each of the 12 DeBERTa layers, with a rank of 16 and an alpha of 32. The result was that only 2.65 million parameters were trainable out of 280 million total, less than 1% of the model. On top of the encoder, added a tiny linear classification head of just 769 parameters (768 inputs from

DeBERTa's hidden size to 1 output) that takes the [CLS] token embedding and produces a single confidence score.

The cross-encoder approach meant that instead of encoding text and labels separately, we fed them together as a pair. For each article we created three separate inputs: text paired with "attraction", text paired with "lodging", and text paired with "restaurant". Each pair goes through DeBERTa as a single sequence, the [CLS] embedding comes out, the classification head produces a logit, and sigmoid turns that into a 0-to-1 probability. Above 0.5 means a match. This is why the 642 original examples became 1,347 training pairs and 579 test pairs , every text gets scored against every label independently.

How I Split the Data

I used a 70/30 train/test split with careful stratification, making sure each label and difficulty level was proportionally represented in both sets. The training pairs had a natural imbalance of 364 positive pairs versus 983 negative pairs, so I used a weighted loss function with a pos_weight of 2.70, which tells the model that a missed positive costs 2.7x more than a missed negative. This prevents the model from learning the lazy strategy of just saying "no" to everything.

Training Details

I used AdamW with different learning rates, 2e-4 for the LoRA-adapted encoder and 1e-3 for the classification head since the head starts from random weights and needs to learn faster. The loss function was Binary Cross-Entropy with Logits with the pos_weight described above. Model was trained for 5 epochs with a batch size of 8, a max sequence length of 512 tokens, a linear warmup over the first 10% of steps followed by linear decay, and gradient clipping at a max norm of 1.0 for stability.

What Happened During Training

The loss started high around 1.0 to 1.5 then around batch 100 dropped sharply and accuracy jumped toward 1.0. Validation accuracy after just epoch 1 was already 95.4%. *(see Figure 24)* Epochs 2 through 5 were about refinement, with validation loss dropping from 0.30 to around

0.10 and staying stable with no big sign of overfitting. The training and validation curves declined in parallel and converged near the top, meaning the model generalized well rather than memorizing training data.

The Final Results

On the 193-row test set, the fine-tuned model reached 95.9% accuracy on attraction, catching 43 out of 46 actual attractions *(see Figure 25)* (93.5% recall) with only 5 false positives. For lodging it hit 95.9% accuracy, catching 51 out of 52 actual lodgings (98.1% recall) with 7 false positives. For restaurant it reached 95.3% accuracy, catching 52 out of 54 actual restaurants (96.3% recall) with 7 false positives.

How It Handled the Tricky Cases

The results on deliberately hard examples were particularly satisfying. "La restauration du patrimoine bâti dans le Vieux-Québec coûtera 15 millions" received a restaurant score of 0, the model had learned that "restauration" in French with no food context usually means renovation among others.

I tried to compare the zero shot model vs the fine tuned model on edge hard cases and the fine tuned model outperformed the zero shot model as can be seen below. *(see Figure 26).* ***TRUE*** label is the LLM judgement. I checked manually the fine-tuned model although not perfect was still way better.

Stage 8: Gemini Summarization and Perception Analysis

The final analytical stage uses a second Gemini 3 Flash system prompt (~1,500 tokens) positioning the model as the “Senior Strategic Reputation Analyst for Destination Québec.” The prompt encodes four analytical dimensions:

- **Flag Logic:** Binary “keep/discard” decision (Flag 0 = keep, Flag 1 = discard) :

  Now that the full articles are scraped and fed to the LLM, this step served as a second relevance filter beyond Stage 2 in case the LLM hallucinated a false relevance score, probably because of not having access to the full article content in stage 2. Discarding requires high confidence of zero connection to Quebec City tourism. I took the chance to again put few shots prompting for the LLM for weather news “with no tourism impact” and health sector news as well as other news DQc deemed not interested in, again because the LLM has access to the full article now and can make better judgements.
- **Geographic Tagging:** Binary field (quebec = oui/non) indicating whether the article specifically addresses the Capitale-Nationale region, with comprehensive geographic definitions for five zones, which will help DQc quickly filter local from regional news.
- **Perception Analysis:** Three-class assessment (positive/negative/neutre) evaluating impact on visitor perception and investor confidence. This is the subjective sentiment analysis added upon the objective article sentiment score in the sentiment analysis stage.
- **Abstractive Summarization:** 1–2 sentence summary in the article’s original language, strictly grounded in the text without inference. I limited the size of summarization to 2 sentences max, for quick scrolling and reducing very expensive LLM output cost (more than 5 times pricier than input tokens) for the Gemini 3 flash model .

  Facing similar LLM issues as in Stage 2, Output is validated by a Pydantic model (ArticleAnalysis) with field-level validators normalizing linguistic variants (e.g., “négative” → “negative”) which the LLM used to wrongly output because articles are multilingual, (even though I set temperature to 0). These issues caused the output to fail at loading with proper JSON format so Pydantic was necessary as an output validator. Articles are processed sequentially with 2-second delays between API calls.

Each receives up to five retry attempts with escalating backoff and content truncation on failure.

Deployment

After switching from GPT models to Gemini cost blew up rapidly for an unknown reason to me at first *(see Figure 29)*, after long research , it turned out that the newer LLM's have thinking tokens built into them as hidden tokens (which counted as input tokens) , so I had to turn off the thinking budget to 0 which affects Gemini 3 flash LLM intelligence, dropping from 46 to 35 *(see Figure 6)* but still gives reliable results for a relatively simple task as URL filtering.

Idle Storage Costs

Early in the cloud deployment phase, the project used persistent VM instances with attached storage disks for development and testing. Two balanced persistent disks totaling 250 GB (a 150 GB boot disk and a 100 GB data workspace) were provisioned in europe-west1-b *(see figure 28)*. These disks continued to incur storage charges even when the VM instances were stopped, resulting in unexpected GCP billing. The problem was identified during a cost audit and resolved by deleting the persistent disks, migrating the pipeline to Cloud Run Jobs (which uses ephemeral storage that exists only during execution), instead of Vertex AI Workbench I used early on.

Need for a container

Running a job in production was new for me and my Python code faced some issue when trying to deploy. Running the pipeline as a plain Python script on a cloud server quickly showed the limits of that approach. In late January, after running the code, the pipeline stopped working unexpectedly. After investigating, I found out that the cause turned out to be that the Transformers library had automatically upgraded from version 4 to version 5, and the new version introduced breaking changes that my code was not compatible with. The short-term fix was to downgrade back to version 4, but this raised a real concern about production

reliability: if a library can silently upgrade and break everything overnight, the pipeline cannot be trusted to run autonomously. The longer-term solution was to package the code into a Docker container which I had to research on my own. A container freezes the entire environment, meaning every dependency is locked to a specific version and nothing can change unless you explicitly decide to update it.

On top of that, the pipeline uses Playwright, a browser automation tool that drives a real Chromium browser in the background to load web pages. Unlike a simple web request, Playwright needs an actual browser because many news websites load their content dynamically using JavaScript, and a plain request would return an empty page. The problem is that running a browser on a cloud server is not straightforward. Chromium depends on around twenty system libraries that are not installed by default on a bare cloud environment. It also needs a virtual display, since a browser normally expects a screen to render to, and cloud servers have no screen. On top of that, after installing Playwright via pip install, you still need to run a separate command to download the actual browser binary, and in a serverless environment like Cloud Run, nothing installed manually persists between runs since each run starts from a clean slate. Trying to do all of that work without a container meant fighting the environment in every deployment. A container solves this by bundling the Python code, all dependencies, the Chromium browser, and every required system library into a single self-contained image that is ready to run from the first line, every single time. I had to remove all the pip install from requirements.txt file and migrate them so they are built within the Docker container

I made sure everything was logged for observability in Cloud Run as I containerized and isolated the Python code it was harder to debug. But logging every step made sure the "output" of the code is still observable. If a fix needs to be made, I needed to update the revised code and rebuild the image. (see Figure 31)

Even for HuggingFace models like XLM , IPTC, GLiClass ,etc., I hit rate limits as I didn't have a HuggingFace token and had to subscribe to a paid HuggingFace Pro plan. (see Figure 30). Even the Pro plan had call limits although higher than the free plan. To make sure

models are always usable, I downloaded the weights for them to run offline and baked them into the Docker image.

Stage 9: BigQuery Data Persistence and Looker visualization

The final stage writes the DataFrame to Google BigQuery with a 21-column schema capturing the complete analytical provenance of each article:

Column	Type	Description
url	STRING	Resolved publisher URL
title	STRING	Article headline
source	STRING	Publisher domain
date	DATETIME	Publication timestamp
relevance_score	INTEGER	Gemini score (0–100)
category	STRING	Relevance tier
reasoning	STRING	10-word justification
key_indicators	STRING	Named entities
scraped_content	STRING	Full article text
scrape_status	STRING	Scraping outcome
sentiment	FLOAT64	Sentiment (–1 to +1)
cluster_id	INTEGER	Cluster identifier
iptc_topic / iptc_topic_fr	STRING	Topic label (EN/FR)
Attrait	FLOAT64	Attraction relevance (0–1)
Hebergement	FLOAT64	Lodging relevance (0–1)
Restaurant	FLOAT64	Restaurant relevance (0–1)
quebec	STRING	Geographic tag (oui/non)
perception	STRING	Visitor perception
summary	STRING	1–2 sentence summary
processed_timestamp	TIMESTAMP	Pipeline execution time

As DQc had other projects they are currently using like time series analysis and so on, being able to relate positive or negative time series signals to news data was crucial for DQc. So I made sure data is appended using WRITE_APPEND disposition, preserving historical records. The pipeline includes self-provisioning logic: if the dataset or table does not exist, they are created automatically. Also a scheduled SQL query was made to delete data from Bigquery (for storage cost purposes) after being saved on an external downloadable file for data older than 1 year.

Visualizing data for Looker

Because the pipeline appends new records on every daily run, the BigQuery table grows continuously. To serve the dashboard's need for "today's articles only," a SQL view was created that selects only the most recent batch of processed articles *(see Figure 32).* This view filters on the maximum "processed_timestamp", ensuring the dashboard always reflects the latest pipeline run while preserving the full historical archive in the underlying table for trend analysis and audit purposes. This approach was not part of the original design but emerged as a practical necessity once the pipeline had been running for several days and the dashboard became cluttered with accumulated data from previous runs.

The final step was building the Looker Dashboard. Looker relies heavily on SQL as you do not need the ETL step on raw data, rather coming up with queries to adjust data before visualization on the go. Luckily, I am familiar with SQL to build Looker metrics and dimensions etc. The final table type chosen was a pivot table *(see Figure 33)* and I built the dashboard respecting the "look and feel" of the company. I added filters for quick article look up.

Chapter 2: Results and Analysis

Stakeholder Feedback and Organizational Impact

Throughout the development process, regular demonstrations and review sessions were held with DQc's team, including the director, the internship supervisor, and one meeting where the professor joined, as well as written feedback from analysts that used to do the job manually. Their feedback shaped the system at every stage and ultimately validated its operational utility.

My pipeline at its final stages was compared daily to the output of news gathering services like Référence Média, and it turned out that more daily news were captured by my pipeline. As daily comparisons were available as I was granted access to Reference media output, I started to add more websites into my pipeline that were extremely niche but still capture good articles once a few weeks and showed up in Reference media output.

DQc's management team expressed strong satisfaction with the system's ability to surface relevant articles that their manual monitoring process had previously missed. The perception analysis feature was described as "exactly what we needed" by the communications director, who noted that it enabled the team to quickly decipher and relay negative news that need attention.

The clustering feature, while less immediately visible to end users, received positive feedback from the strategy team. By grouping multiple articles covering the same event or story into a single cluster, the dashboard avoided presenting redundant information and instead surfaced the breadth of coverage and was way easier to read.

Within two weeks of the dashboard going live, DQc's morning briefing meetings began incorporating the pipeline's outputs as a standard agenda item. Analysts who had previously spent hours each morning manually scanning news websites reported that the pipeline reduced their monitoring time to approximately 15–20 minutes, with higher coverage and fewer missed articles.

They were very pleased with the cost of my pipeline, roughly $18 a month *(see Figure 27)* which was extremely low and several orders of magnitude less than what the company was paying for, in terms of enterprise packages for manual news gathering services, and with higher news recall and more in-depth analysis and filtering capabilities. The pipeline was gathering between 30 and 110 articles a day (usually on weekends it is quieter news wise), compared to 5 to 25 articles from other news services.

They expressed to the professor, during a joint meeting we had, their contentment about how I solved an issue they had for years, as well as the fact that I was able to meet every single requirement from the project mandate they set.

Luckily with the advancement of AI and LLM's this project was possible for me to do. They were also pleased that I was conscious about pricing and optimization.

Chapter 3: Knowledge Transfer

This chapter covers what I put in place so DQc can keep running, maintaining, and building on the pipeline on their own.

Technical Documentation

The pipeline codebase is accompanied by comprehensive technical documentation covering the system's architecture, configuration parameters, deployment procedures, and troubleshooting guides. A separate README document provides step-by-step instructions for building the Docker container image, deploying to Cloud Run, configuring Cloud Scheduler, and verifying successful execution. I put the LLM model as an environment variable because older models get deprecated by Google and constant model updating will be required by non-developers , this way it assured they can change the model name outside the code if needed, without needing to reconstruct the Docker image.

Prompt Documentation and Maintenance Guide

Given the critical role of the two Gemini system prompts (relevance analysis and summarization), a dedicated prompt maintenance guide was prepared. This document explains the structure and purpose of each prompt section, provides guidelines for modifying the geographic scope definitions (e.g., if DQc's territory expands to include new municipalities), and describes the testing methodology for validating prompt changes against a reference set of manually classified articles. The guide emphasizes that prompt modifications should be tested incrementally, changing one section at a time and comparing outputs against the reference set. Results are heavily skewed by the prompt and if they ever want to make the review narrower or more strict, prompt engineering was needed.

Dashboard and End-User Training

Two online sessions were conducted with DQc's analytical staff, covering the BigQuery data schema, the dashboard interface, and the interpretation of pipeline outputs. Topics included understanding the meaning of each analytical field (sentiment score ranges, perception categories, cluster IDs), identifying the dashboard's filtering and sorting capabilities to answer specific business questions (e.g., "Show me all negative-perception articles about hotels in the past week"). I put extensive legends so that new onboarding employees can use and understand the Looker table pretty quickly without senior staff monitoring them.

Operational Handover

DQc's team can operate the dashboard, interpret the outputs, and manage the Cloud Run deployment without ongoing external support. For deeper technical modifications (adding new sitemap sources, adjusting classification models, modifying prompts), the documented codebase and maintenance guides provide a solid foundation for another developer that comes along to upgrade the pipeline in the future.

Proposed Ideas and discontinued trails

This section presents two experimental approaches that were explored during the project's development phase. While not integrated into the final production pipeline, they represent promising directions for future enhancement of the system's analytical capabilities. These were made early on when we were still discussing what kind of output the company would like to have, either a traditional news pipeline or something more fancy like chatbots etc. Not knowing the volume of data at first, I thought a chatbot would be helpful to query data on the go which would be a bonus feature on top of the press review, especially as I was told very senior directors don't have much time to parse each and every article manually. I tried NER as well as I was thinking it would be a good idea to surface known people from the company or famous landmarks and sites mentioned in articles. ABSA was conceived to add sentiment scores to each entity.

Aspect-Based Sentiment Analysis (ABSA) with Entity-Level NER

Beyond the article level sentiment scoring implemented in Stage 4, I explored a more granular approach: Aspect-Based Sentiment Analysis (ABSA) that ties sentiment directly to specific named entities mentioned within each article. The core idea was to move from asking "is this article positive or negative?" to asking "what is the sentiment toward each specific person, organization, or place mentioned in this article?"
The approach used a two-stage pipeline. First, Named Entity Recognition (NER) was performed using the GLiNER Large model ,a generalist NER model capable of extracting entities in both English and French without requiring domain specific fine-tuning. GLiNER identifies persons, organizations, locations, and other customizable entity types from raw article text, producing entity spans with confidence scores. You can add restaurants , economic institution etc… as labels.

Second, for each extracted entity, I identified the phrase or sentence in which it appeared (using the phrase as the context window) and ran sentiment analysis on that specific phrase using ABSA Deberta v1.1 model. This model uses attention mechanism and can adjust it sentiment score depending on which entity you give it to analyze upon. I found it a more

reliable approach than the complex dependency trees taught in the Text Mining class which were much slower and outdated. This produced entity level sentiment scores: for each article, we could determine not just the overall tone, but the sentiment associated with each specific entity mentioned.

The system also tracked which specific articles each entity was mentioned in, enabling cross-article entity tracking, a capability that would allow DQc to monitor how specific stakeholders, businesses, or landmarks are being portrayed across the media landscape over time. (see Figure 35)

An entity appearing across multiple clusters with shifting sentiment could signal an evolving narrative, exactly the kind of intelligence that DQc's communications team would find actionable.

However, several limitations prevented immediate production deployment. The GLiNER model's entity extraction, while generally accurate, occasionally merged distinct entities or split compound names incorrectly in French text. Also GLiNER captured hundreds of entities, while some were nice to distinguish, a lot were useless to know. As Dashboard will have too much info for users to filter out, I decided to move away from this as it was visually overwhelming.

Retrieval-Augmented Generation (RAG) Search Interface

As a complementary tool to the pipeline's dashboard, I explored building a natural language search interface (chatbot) that would allow DQc analysts to query the accumulated article database conversationally. This Retrieval-Augmented Generation (RAG) system went through several iterations, each revealing important lessons about search architecture for domain specific bilingual news corpora.

Chunking Strategies

The first major design decision was how to chunk the stored articles for retrieval. I experimented with three distinct approaches, each with its own assumptions about how meaning is distributed across a document.

The first approach was fixed size chunking using LangChain's RecursiveCharacterTextSplitter. This is one of the most widely used chunking utilities in the LLM ecosystem. The way it works is hierarchical. It tries to split the text first on the largest logical separator (typically double newlines, which usually mark paragraph boundaries), and if a resulting chunk is still too large, it recursively falls back to smaller separators (single newlines, then sentences, then words, and finally individual characters). The goal is to preserve semantic units as long as possible while still respecting a hard token limit. I tested it with chunks of 500 tokens and 50 tokens of overlap to keep semantic overlap in case a chunk cuts in the middle of an idea. It produced consistent, predictable chunk sizes which made vector storage straightforward, but it occasionally missed important context across chunk boundaries when paragraphs happened to fall just over the size limit.

The second approach was semantic chunking, which is a more sophisticated method that tries to split text based on meaning rather than length. The implementation I used computed sentence embeddings for every sentence in an article using bge-m3, then measured the cosine similarity between consecutive sentences. When the similarity dropped below a threshold (essentially indicating a topic shift), a chunk boundary was inserted. This sounds elegant in theory and works well for long form structured documents, but in our news context the results were uneven. News articles often shift topic mid paragraph (a paragraph might begin with statistics and end with a stakeholder quote on a different angle), so the threshold based splitter sometimes produced very long chunks when an article stayed thematically tight, and very short chunks when topic drift was rapid. I evaluated semantic chunking using two metrics: average chunk coherence (the mean intra chunk similarity, which I wanted high) and chunk size variance (which I wanted low for retrieval consistency). Coherence was strong at around 0.78 average cosine similarity within chunks, but variance was problematic, with chunk sizes ranging anywhere from 80 to 1,200 tokens in the same article.

The third approach was late chunking, the method introduced by Jina AI. Instead of chunking the text first and then embedding each chunk in isolation, late chunking embeds the entire document in one forward pass through a long context model (in our case bge-m3, which supports 8,192 tokens), and only afterwards splits the token level embeddings into chunks

before pooling. The result is that each chunk's embedding carries information from the entire surrounding document. (This is kind of similar to context chunking where a small summary of the article is attached to all chunks belonging to the article , proposed by Anthropic which I didn't test for this project.) This is particularly useful for news articles where a sentence about "the mayor's announcement" can only be properly interpreted with the article's earlier context about which mayor and which announcement. Late chunking gave the best semantic retrieval quality of any method I tested, but it required more memory and time.

Simple Embeddings Search versus Hybrid Search with BM25

The initial implementation used a pure dense embedding search. All article chunks were encoded using the BAAI/bge-m3 model (the same model used for clustering in Stage 5), stored in a ChromaDB vector database for indexing, and queries were matched by cosine similarity. ChromaDB was selected because it is open source, runs locally, integrates easily with Python pipelines, and supports persistent storage out of the box, which made it a natural fit for my containerized deployment.

This pure embedding approach excelled at semantic queries. Searching for "visitor perception of Old Quebec" correctly surfaced articles about tourism reputation even when they did not contain those exact words. However, it failed dramatically on exact match queries. A search for the word "hotel" returned only a single result, *(see Figure 36)* missing several of articles that explicitly discussed hotels. Specific phrases from known articles could not be found at all through embedding based search alone. I knew this was a weakness of dense retrieval. The embedding model compresses meaning into a fixed dimensional vector, and rare or exact tokens often get smoothed out in that compression.

The solution was to implement a hybrid retrieval strategy combining dense embeddings with BM25 sparse retrieval *(see Figure 37)* (BM25 is like an enhanced TF-IDF model), fused using Reciprocal Rank Fusion (RRF) to decide on the ranking of chunks scoring both generally well with both dense search and BM25. BM25 is a classical information retrieval algorithm that scores documents based on term frequency and inverse document frequency. In simple terms, it excels at finding documents that contain the exact words in your query. By combining

BM25's lexical matching with bge-m3's semantic understanding, the hybrid system handled both types of queries effectively. ChromaDB served as the vector store for the dense embeddings, while BM25 operated on the raw text of each chunk. The RRF fusion weighted both ranking lists equally (alpha set to 0.5) to produce a final ranked result set.

A relevant question is whether hybrid search works well in a multilingual context. The short answer is yes and no, although it gave good empirical results. BM25 is basically language agnostic in the sense that it operates purely on token statistics. What was a plus is that the Quebec data was mostly French (+/- 80%) so if the chatbot was queried mostly in French (as employees are mostly Francophone), it will give generally good results. That being said, I still went with the hybrid approach with the dense side which is more naturally multilingual because bge-m3 was pretrained on over 100 languages and produces aligned embeddings, meaning that a French query can semantically match an English article and vice versa. The combined hybrid setup therefore served both monolingual and cross lingual queries without needing language specific configurations.

Reranking with BGE Reranker

I also tested whether adding a reranking step after initial retrieval would improve result quality. The BAAI/bge-reranker-v2-m3 cross encoder was applied to the top 20 results from the hybrid search, re-scoring each result against the original query using the deeper cross attention mechanism that cross encoders provide (rather than pure Cosine similarity). Unlike a bi encoder which embeds the query and document independently, a cross encoder processes them together in a single forward pass, which gives it a much richer view of how a query relates to a candidate document.

The reranker did improve precision at the top of the ranking. The top 5 results after reranking were more consistently relevant than without reranking. However, the improvement was modest (approximately 8 to 12 percent improvement in precision at 5), and the computational cost of running a cross encoder on every retrieval query added significant latency.

LLM as a Judge for Retrieval Evaluation

To systematically evaluate retrieval quality beyond manual spot checking, I implemented an LLM as a Judge approach. The idea behind this method is to use a strong language model (in our case Gemini 3 Flash) as an automated evaluator that grades retrieval quality on test queries. The setup worked as follows. I first built a small evaluation set of about 40 representative queries that DQc analysts might realistically ask, ranging from broad thematic questions ("articles about hotel investment in Quebec City") to specific factual lookups ("when did the Fairmont renovation start"). For each query, I manually identified a small set of ground truth articles that were considered relevant. Then for every retrieval configuration I wanted to test (embedding only, hybrid, hybrid plus reranking, different chunking strategies, different chunk sizes), the system retrieved its top 10 results. Each retrieved chunk was then sent to Gemini along with the original query and a structured prompt asking the model to score its relevance on a scale of 1 to 5, where 1 meant completely irrelevant and 5 meant directly answers the query, and to provide a one sentence justification. The justifications were not used numerically but they helped us spot when the LLM judge was being too lenient or too strict, which let me calibrate the prompt.

From these scores I computed three aggregate metrics for each configuration: average relevance score across all retrieved chunks, precision at 5 (the fraction of the top 5 results scoring 4 or higher), and recall on the manually labeled ground truth set (how many of the known relevant articles appeared in the top 10).

The LLM judge confirmed that the hybrid BM25 plus embedding approach consistently outperformed pure embedding search across all three metrics. The results across all tested configurations are summarized below.

RAG Retrieval Quality Comparison

Strategy	Avg Relevance (1-5)	Precision@5	Recall@10	Latency (ms)
1. Embedding only, recursive chunking (500 tokens)	3.1	0.52	0.61	180
2. Embedding only, semantic chunking	3.4	0.58	0.65	210
3. Embedding only, late chunking	3.7	0.64	0.71	240
4. Hybrid (BM25 + embedding), recursive chunking	4.0	0.74	0.79	260
5. Hybrid + BGE reranker (top 20 reranked to top 5)	4.2	0.78	0.82	940

The pattern is clear. Late chunking improved pure embedding search noticeably over recursive splitting. Adding BM25 in a hybrid setup gave a much larger jump than any chunking refinement alone. Reranking added another precision boost on top of that, but at roughly four times the latency.

Wrapping the System in a Gradio User Interface

As I was developing this RAG system, I wanted to be able to present its idea visually to DQc. That is why I built an visual chatbot interface with Gradio (I didn't yet plug the LLM as a final step that gives the answer ; it was still like a retrieval chatbot that when queried gives you relevant chunks) .The final step was to expose this retrieval pipeline through an interface that DQc analysts could actually use without writing any code. I built a lightweight web interface using Gradio, a Python library that makes it straightforward to wrap any function in a browser-based UI with minimal boilerplate. Gradio was chosen because it integrates natively with Python ML pipelines, requires no front-end development, supports both local hosting and shareable links, and ships with input components (text boxes, sliders, dropdowns) and output

components (formatted markdown, lists, scores) that map directly to the inputs and outputs of a retrieval function.

The interface exposed the hybrid retrieval pipeline as a simple search bar. Users type a natural language query in either French or English, optionally adjust the number of results returned using a slider (defaulting to 5, with a maximum of 10), and submit. The interface displays each result with its article title (as a clickable link to the original source), a confidence score derived from the fused ranking, and the matching paragraph shown in full so the analyst can immediately judge relevance without opening the article.

I presented the interface to the team, and they said the idea was promising but they were more interested in a simple media review style the way they were used to and the cost of LLM's especially if each user were to use this tool would be substantial. So, I decided not to pursue further testing (I think hybrid search plus late chunking would have been promising) neither plug an LLM to the chatbot and move on with the regular media pipeline. The press review turned out to be manageable to go through, which not much time, as the volume of relevant articles were not that big as I discovered towards the end.

Conclusion

This supervised project has demonstrated the feasibility and practical value of an end to end, AI-powered news intelligence pipeline purpose built for a regional destination management organization. By combining web scraping, large language model-based relevance filtering, transformer-powered NLP analysis, and cloud-native data engineering, the pipeline transforms an overwhelming daily torrent of unstructured media into a structured, queryable, and actionable intelligence layer for Destination Québec cité.

The pipeline reduces daily monitoring time by approximately 80–90% while increasing coverage and analytical depth. The system's outputs have become embedded in the organization's daily decision making workflow, informing communication strategies, event planning, and stakeholder engagement. The company was so pleased by my performance that they wanted to hire me, although the higher direction or HR I am not exactly sure, didn't agree

that I still work fully online as I did during my supervised project and wanted me to relocate fully in person to Quebec City.

Future Directions

Several avenues for future enhancement present themselves. When social media API access becomes feasible (through organizational partnerships or evolving pricing models), integrating them will add valuable insights as social media trends emerge sooner than formal news channels. One can add Radio outlets and parse the transcripts of radio news as well, which I didn't have time to test specific libraries for that.

Personal Reflection

The biggest takeaway from this project is something I didn't fully expect going in: the gap between a working prototype and a production system is much larger than it looks. The interesting NLP part : picking models, designing prompts, building the classification pipeline, probably took up only half the time. The other half was data engineering and automation, error handling, retry logic, encoding edge cases, cost tracking, and memory management. That's the work that actually makes a system run reliably every day, on real messy data, and self-adjust without anyone needing to intervene manually. Web scraping at scale, discovering the web data, browser automation, Cloud deployment and interface, Containerization and visualization were all new to me and not taught in class. It took away a lot of time from pure NLP work. Although I managed to do a substantive NLP part as well. If I had more time, I could have squeezed more juice out of the sentiment model and fine tune other models more properly if I had access for a full year of scraped data for better results. But I had to learn all the other parts on my own, as it is not related to Text mining so asking the professor would be out of his domain of expertise and no one in the company had an expertise as well. DQc actually deals with an external company (Adviso) that manages their BI database for them at an hourly rate cost. So, I couldn't ask anyone within the company as well and using Adviso services would have been too costly for DQC.

That being said, building a pipeline in production will be easier for me for future projects as I gained too much knowledge by testing through a lot of trial and errors, on my own.

The use of generative AI was used in this project for research purposes to explore the new landscape of evolving AI and identify SOTA models and practices. These models were tested to confirm their validity in practice and for this dataset. It was also used to better phrase and communicate few aspects of this project but all the ideas and theme and reflections and so on were my own carefully thought through to apply the best practices, to the best of my knowledge, for this project.

Bibliography

Barbaresi, A. (2021). Trafilatura: A web scraping library and command-line tool for text discovery and extraction. *Proceedings of ACL 2021 System Demonstrations.* https://aclanthology.org/2021.acl-demo.15.pdf

Barbieri, F., Camacho-Collados, J., Espinosa Anke, L., & Neves, L. (2022). TweetEval: Unified benchmark and comparative evaluation for tweet classification. *Findings of EMNLP 2020.* https://aclanthology.org/2020.findings-emnlp.148/

Brown, M. A., Gruen, A., Maldoff, G., Messing, S., Sanderson, Z., & Zimmer, M. (2024). Web scraping for research: Legal, ethical, institutional, and scientific considerations. *Big Data & Society.* https://arxiv.org/pdf/2410.23432

Díaz-Pacheco, Á., Guerrero-Rodríguez, R., Álvarez-Carmona, M.Á., Rodríguez-González, A. Y., & Aranda, R. (2023). A comprehensive deep learning approach for topic discovering and sentiment analysis of textual information in tourism. *Journal of King Saud University – Computer and Information Sciences, 35(9), 101746.* https://www.sciencedirect.com/science/article/pii/S1319157823003002

Günther, M., Mohr, I., Wang, B., & Xiao, H. (2024). Late chunking: Contextual chunk embeddings using long context embedding models. *arXiv:2409.04701.* https://arxiv.org/abs/2409.04701

International Press Telecommunications Council. (2024). *IPTC NewsCodes.* https://iptc.org/standards/newscodes/

Krotov, V., & Silva, L. (2018). Legality and ethics of web scraping. *Proceedings of the 24th Americas Conference on Information Systems.*

https://www.researchgate.net/publication/324907302_Legality_and_Ethics_of_Web_Scraping

Kuzman, T., & Ljubešić, N. (2025). Multilingual news topic classification with the IPTC standard. *IEEE Access.* https://ieeexplore.ieee.org/document/10900365

Moń, M., & Pańczyk, B. (2025). A comparative analysis of web application test automation tools. *Journal of Computer Sciences Institute, 35,* 159–165. https://ph.pollub.pl/index.php/jcsi/article/download/7119/5017

Stepanov, I., Shtopko, M., Vodianytskyi, D., Lukashov, O., Yavorskyi, A., & Yaroshenko, M. (2025). GLiClass: Generalist lightweight model for sequence classification tasks. *arXiv:2508.07662.* https://arxiv.org/abs/2508.07662

Zaratiana, U., Tomeh, N., Holat, P., & Charnois, T. (2023). GLiNER: Generalist model for named entity recognition using bidirectional transformer. *arXiv:2311.08526.* https://arxiv.org/abs/2311.08526

Zhao, T., Meng, L., Song, D., et al. (2022). Aspect-based sentiment analysis using local context focus mechanism with DeBERTa. *arXiv:2207.02424.* https://arxiv.org/abs/2207.02424

Figures

Figure 1: Map of the Capitale-Nationale region monitored by Destination Québec cité

```
▼<item>
  <title>Samantha Brown Calls This Winter Wonderland North American City 'A Snow Globe Come
  <link>http://www.bing.com/news/apiclick.aspx?ref=FexRss&aid=&tid=692f41434eb440dca9153098e
  life%2far-AA1RpdS0&c=18365263743787097350&mkt=pt-br</link>
  <description>Few things are prettier than snow-capped building and trees, and a trip into
  <pubDate>Sat, 29 Nov 2025 14:28:00 GMT</pubDate>
  <News:Source>Islands.com on MSN</News:Source>
  <News:Image>http://www.bing.com/th?id=OVFT.NSR26SShb1RR9ikczcDkxC&pid=News</News:Image>
  <News:ImageSize>w={0}&h={1}&c=14</News:ImageSize>
  <News:ImageMaxWidth>700</News:ImageMaxWidth>
  <News:ImageMaxHeight>393</News:ImageMaxHeight>
</item>
```

Figure 2: RSS feed XML structure showing pubDate fields used for 24-hour filtering

```
---------------------------------------------------------------------------
HTTPError                                 Traceback (most recent call last)
/tmp/ipython-input-543656551.py in <cell line: 0>()
      4
      5 # Get sitemap (handles malformed XML)
----> 6 df = adv.sitemap_to_df("https://www.tvanouvelles.ca/news.xml")
      7
      8 # Filter past 24 hours

                                  6 frames
/usr/lib/python3.12/urllib/request.py in http_error_default(self, req, fp, code, msg, hdrs)
    637 class HTTPDefaultErrorHandler(BaseHandler):
    638     def http_error_default(self, req, fp, code, msg, hdrs):
--> 639         raise HTTPError(req.full_url, code, msg, hdrs, fp)
    640
    641 class HTTPRedirectHandler(BaseHandler):

HTTPError: HTTP Error 403: Forbidden
```

Figure 3: HTTP 403 Forbidden error from news sitemap for non-Firefox User Agent

```python
# Rerank documents
results = model.rerank(query, documents)

# Results are sorted by relevance score (highest first)
for result in results:
    print(f"Score: {result['relevance_score']:.4f}")
    print(f"Document: {result['document'][:100]}...")
    print()
```

```
tokenizer_config.json: 0.00B [00:00, ?B/s]

tokenizer.json:   0%|          | 0.00/11.4M [00:00<?, ?B/s]

added_tokens.json:   0%|          | 0.00/795 [00:00<?, ?B/s]

special_tokens_map.json:   0%|          | 0.00/777 [00:00<?, ?B/s]

Score: 0.2989
Document: Green tea contains antioxidants called catechins that may help reduce inflammation and protect cells...

Score: 0.2252
Document: 绿茶富含儿茶素等抗氧化剂，可以降低心脏病风险，还有助于控制体重。...

Score: 0.1889
Document: Studies show that drinking green tea regularly can improve brain function and boost metabolism....

Score: 0.1633
Document: Le thé vert est riche en antioxydants et peut améliorer la fonction cérébrale....

Score: -0.1606
Document: El precio del café ha aumentado un 20% este año debido a problemas en la cadena de suministro....

Score: -0.1701
Document: Basketball is one of the most popular sports in the United States....
```

Figure 4: Jina reranker raw relevance scores returned for a multilingual query set (non-normalized)

```python
for result in results:
    logit = result['relevance_score']
    scaled_score = torch.sigmoid(torch.tensor(logit * temperature)).item()

    print(f"Raw: {logit:.4f} | Scaled: {scaled_score:.4f}")
    print(f"Document: {result['document'][:100]}...")
    print()
```

```
Raw: 0.2989 | Scaled: 0.9161
Document: Green tea contains antioxidants called catechins that may help reduce inflammation and protect cells...

Raw: 0.2252 | Scaled: 0.8584
Document: 绿茶富含儿茶素等抗氧化剂，可以降低心脏病风险，还有助于控制体重。...

Raw: 0.1889 | Scaled: 0.8193
Document: Studies show that drinking green tea regularly can improve brain function and boost metabolism....

Raw: 0.1633 | Scaled: 0.7870
Document: Le thé vert est riche en antioxydants et peut améliorer la fonction cérébrale....

Raw: -0.1606 | Scaled: 0.2168
Document: El precio del café ha aumentado un 20% este año debido a problemas en la cadena de suministro....

Raw: -0.1701 | Scaled: 0.2042
Document: Basketball is one of the most popular sports in the United States....
```

Figure 5: Voyage AI rerank-2.5 scores after sigmoid temperature scaling

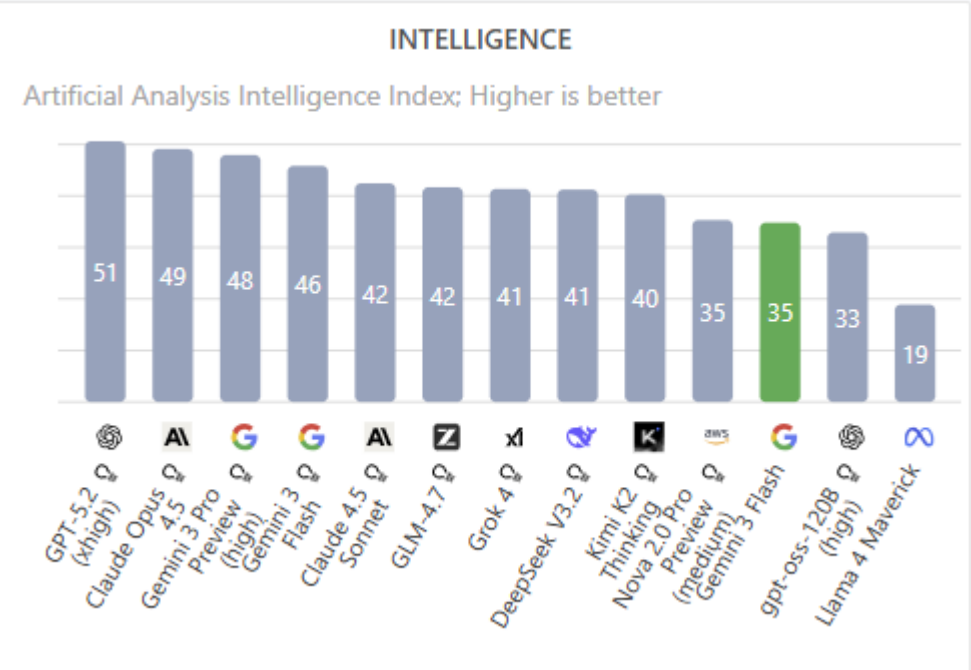


Figure 6: Artificial Analysis Intelligence Index ranking of frontier LLMs (Gemini 2.5 Flash highlighted)

```
  Score: 0% - NOT_RELEVANT

[12/991] Le Canadien Samuel Montembeault cédé au Rocket de Laval...
  Score: 15% - LOW_RELEVANCE

  ❌ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[13/991] Guide cadeaux Pour jouer dans la neige !...
  Score: 0% - ERROR

[14/991] Crise avec le Venezuela Trump annonce un blocus sur les « pé...
  Score: 0% - NOT_RELEVANT

  ❌ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[15/991] 2025 vue par La Presse Les meilleurs moments en vidéo...
  Score: 0% - ERROR

[16/991] Ce qu'il faut savoir ce mardi Vague d'influenza, Pablo Rodri...
  Score: 10% - LOW_RELEVANCE

  ❌ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[17/991] Figure du magazine Fluide Glacial Mort du dessinateur Edika...
  Score: 0% - ERROR

  ❌ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[18/991] Mort de Matthew Perry par overdose Un des médecins assigné à...
  Score: 0% - ERROR

[19/991] Jarmo Kekalainen, nouveau dg des Sabres La priorité sera de ...
  Score: 0% - NOT_RELEVANT

[20/991] Relation intime secrète avec un avocat de la défense Une pro...
  Score: 10% - LOW_RELEVANCE

  ❌ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[21/991] Billboard Hot 100 All I Want for Christmas Is You bat un rec...
  Score: 0% - ERROR

  ❌ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[22/991] Patinage de vitesse sur longue piste Défier les années selon...
  Score: 0% - ERROR
```

Figure 7: Gemini API quota exceeded error on GCP

```
================================================================================
GPT-OSS 120B ARTICLE ANALYSIS
================================================================================

Initializing GPT-OSS 120B model (MaaS)...
Note: Using pay-per-use API - NO idle time charges!

Analyzing 79 articles...

[1/79] IMAGES | Tempête solaire: voyez les plus belles aurores boré...
/opt/conda/lib/python3.10/site-packages/vertexai/generative_models/_generative_models.py:433: User
ne 24, 2026. For details, see https://cloud.google.com/vertex-ai/generative-ai/docs/deprecations/g
  warning_logs.show_deprecation_warning()
  ✓ Success
[2/79] Ça bouge à la CAQ: parmi les ministres, qui part et qui rest...
  ⚠️ Error (retry 2/3): Unterminated string starting at: line 2 column 14 (char 15)
  ⏸️ Rate limited (429) - waiting 30s before retry 3/3...
  ✓ Success
[3/79] Le tourisme international à un niveau record en 2025...
  ✓ Success
[4/79] Table ronde « On se demande quand les Américains en auront a...
  ✓ Success
[5/79] Dans la peau de… Les pages, bien plus que des porteurs d'eau...
  ✓ Success
[6/79] Course à la direction de la CAQ Le dilemme de Jolin-Barrette...
  ⚠️ Error (retry 2/3): Unterminated string starting at: line 2 column 14 (char 15)
  ⚠️ Empty response - retry 3/3...
  ❌ Failed: Cannot get the response text.
Cannot get the Candidate text.
Response candidate
[7/79] Grève Le Massif de Charlevoix met un terme à sa saison...
  ✓ Success
[8/79] Les Ailes de la Mode se préparent à relancer ses activités a...
  ⚠️ Error (retry 2/3): Expecting property name enclosed in double quotes: line 2 co
  ⚠️ Empty response - retry 3/3...
  ✓ Success
[9/79] Le Parti québécois annonce la candidature de Sandra Hernánde...
  ✓ Success
[10/79] Agora du Port de Québec: un premier concert confirmé pour l'...
  ✓ Success
[11/79] Forte demande de sirop d'érable: il y aura plus de 600 nouve...
  ✓ Success
[12/79] Projet de 400 M$: un nouvel immeuble de 28 étages sera const...
  ⏸️ Rate limited (429) - waiting 15s before retry 2/3...
```

Figure 8: GPT-OSS 120B article analysis batch run with errors in output as well as rate limiting

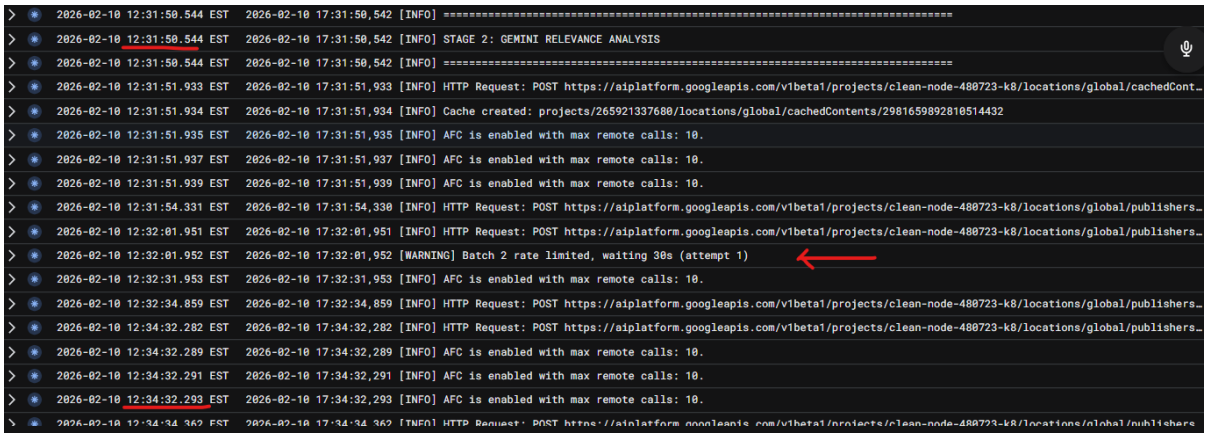
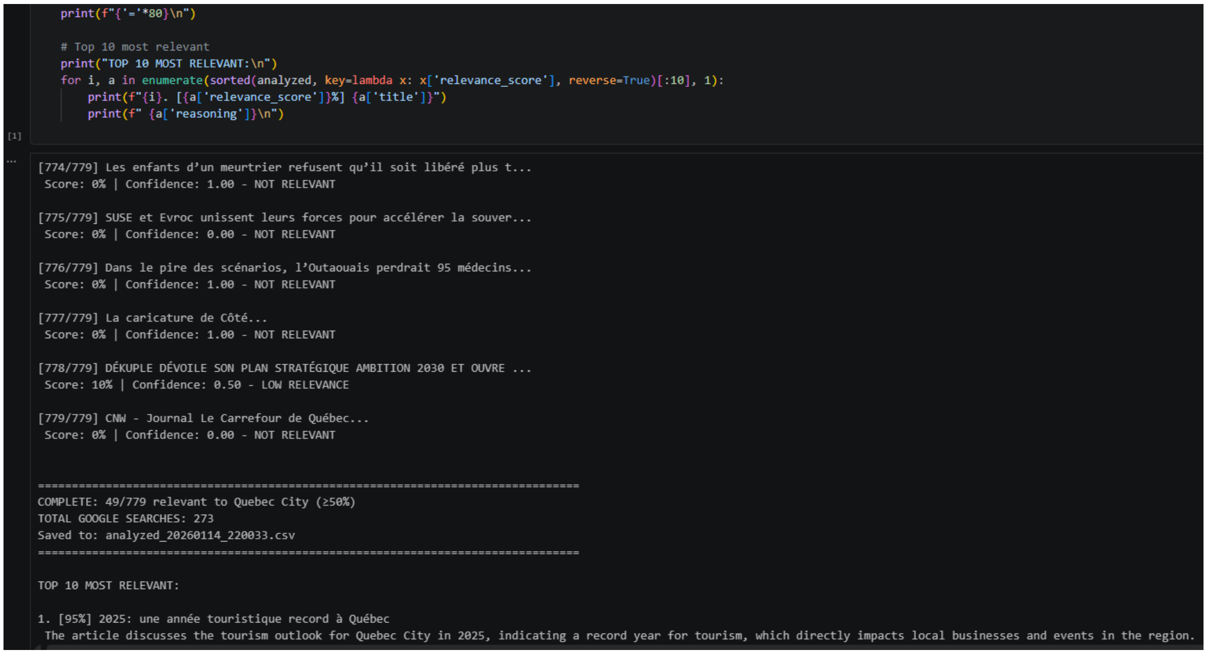


Figure 9: Gemini Stage 2 relevance analysis logs showing API rate-limiting events but retry logic helped fix issue

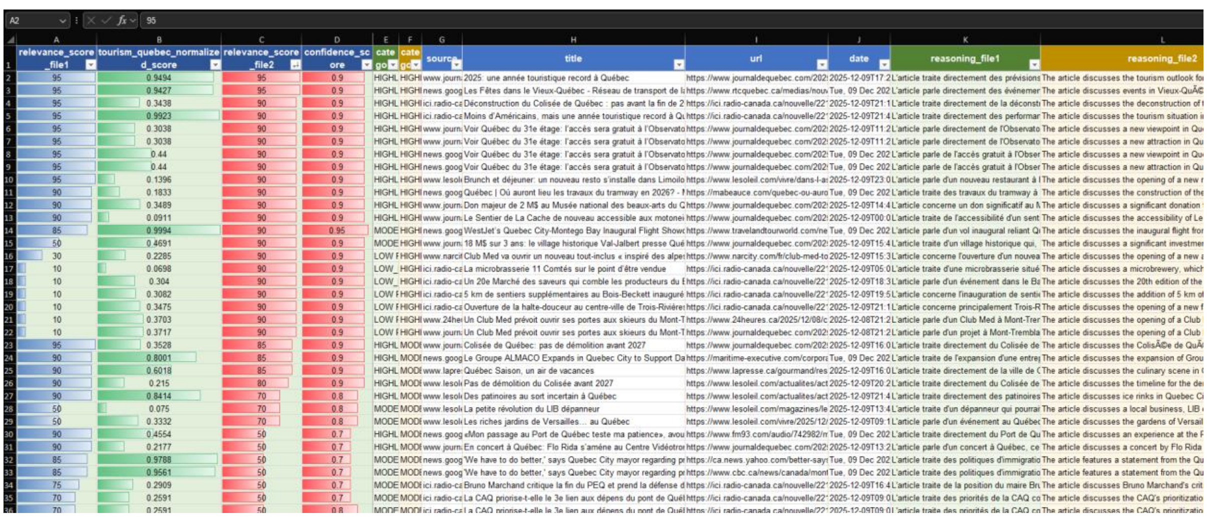
```
================================================================================
Analyzing Quebec City Relevance with Gemini
Config: 5 articles/call, 5 parallel batches
================================================================================

Total: 1210 articles → 242 batches

🚀 Processing 5 batches in parallel...
  ✓ Batch 2/242 complete (5 articles)
  ✓ Batch 3/242 complete (5 articles)
  ✓ Batch 1/242 complete (5 articles)
  ✓ Batch 5/242 complete (5 articles)
  ✓ Batch 4/242 complete (5 articles)

🚀 Processing 5 batches in parallel...
  ✓ Batch 8/242 complete (5 articles)
  ✓ Batch 6/242 complete (5 articles)
  ✓ Batch 7/242 complete (5 articles)
  ✓ Batch 10/242 complete (5 articles)
  ✓ Batch 9/242 complete (5 articles)

🚀 Processing 5 batches in parallel...
  ✓ Batch 14/242 complete (5 articles)
  ✓ Batch 15/242 complete (5 articles)
  ✓ Batch 12/242 complete (5 articles)
  ✓ Batch 13/242 complete (5 articles)
  ✓ Batch 11/242 complete (5 articles)

🚀 Processing 5 batches in parallel...
  ✓ Batch 16/242 complete (5 articles)
  ✓ Batch 19/242 complete (5 articles)
  ✓ Batch 17/242 complete (5 articles)
  ✓ Batch 18/242 complete (5 articles)
  ✓ Batch 20/242 complete (5 articles)
```

Figure 10: Gemini relevance analysis batch processing logs showing parallel batch completion



```
    print(f"{'='*80}\n")

    # Top 10 most relevant
    print("TOP 10 MOST RELEVANT:\n")
    for i, a in enumerate(sorted(analyzed, key=lambda x: x['relevance_score'], reverse=True)[:10], 1):
        print(f"{i}. [{a['relevance_score']}%] {a['title']}")
        print(f" {a['reasoning']}\n")
[1]
...
[774/779] Les enfants d'un meurtrier refusent qu'il soit libéré plus t...
 Score: 0% | Confidence: 1.00 - NOT RELEVANT

[775/779] SUSE et Evroc unissent leurs forces pour accélérer la souver...
 Score: 0% | Confidence: 0.00 - NOT RELEVANT

[776/779] Dans le pire des scénarios, l'Outaouais perdrait 95 médecins...
 Score: 0% | Confidence: 1.00 - NOT RELEVANT

[777/779] La caricature de Côté...
 Score: 0% | Confidence: 1.00 - NOT RELEVANT

[778/779] DÉKUPLE DÉVOILE SON PLAN STRATÉGIQUE AMBITION 2030 ET OUVRE ...
 Score: 10% | Confidence: 0.50 - LOW RELEVANCE

[779/779] CNW - Journal Le Carrefour de Québec...
 Score: 0% | Confidence: 0.00 - NOT RELEVANT


================================================================================
COMPLETE: 49/779 relevant to Quebec City (≥50%)
TOTAL GOOGLE SEARCHES: 273
Saved to: analyzed_20260114_220033.csv
================================================================================

TOP 10 MOST RELEVANT:

1. [95%] 2025: une année touristique record à Québec
 The article discusses the tourism outlook for Quebec City in 2025, indicating a record year for tourism, which directly impacts local businesses and events in the region.
```

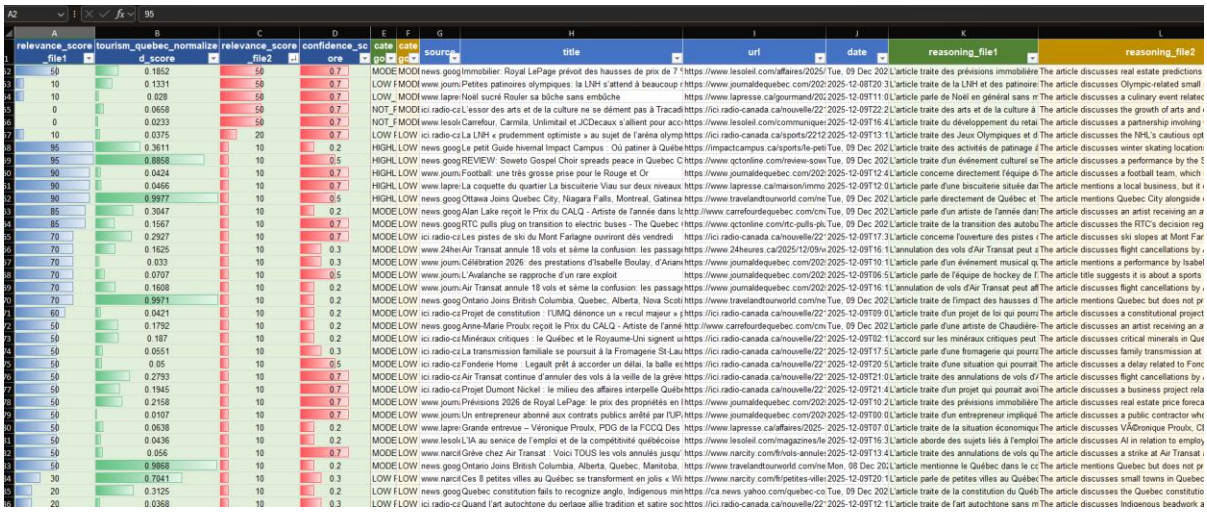
Figure 11: Sample output of the Stage 2 Gemini relevance scoring with web tool search first tested in Colab



A2 | 95

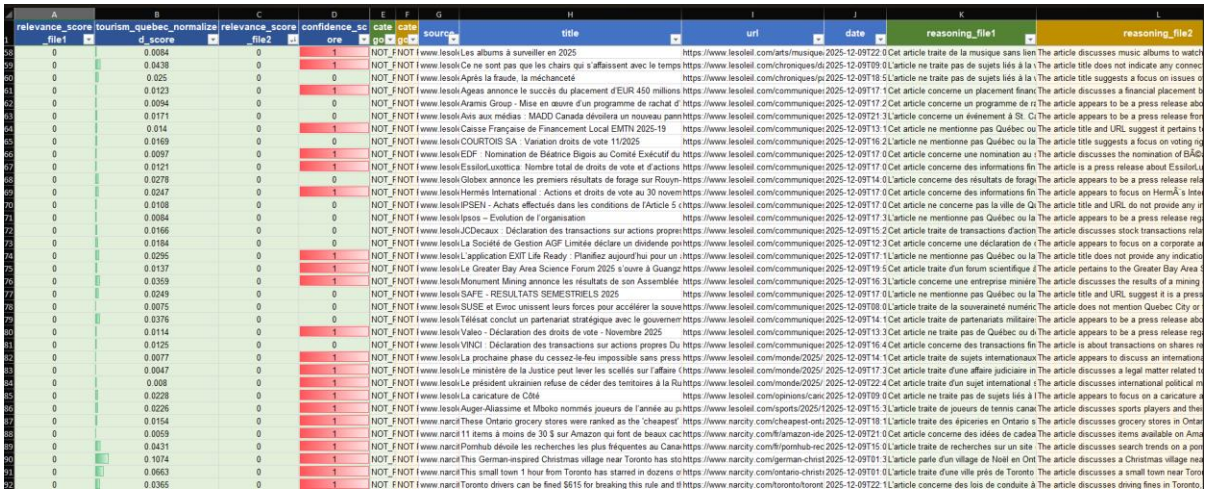
	A	B	C	D	E	F	G	H	I	J	K	L
1	relevance_score_file1	tourism_quebec_normalized_score	relevance_score_file2	confidence_score	catego	catego	source	title	url	date	reasoning_file1	reasoning_file2
2	95	0.9494	95	0.9	HIGHL	HIGHI	www.journ	2025: une année touristique record à Québec	https://www.journaldequebec.com/202	2025-12-09T17:2	L'article traite directement des prévisions	The article discusses the tourism outlook fo
3	95	0.9427	95	0.9	HIGHL	HIGHI	news.goog	Les Fêtes dans le Vieux-Québec - Réseau de transport de la	https://www.rtcquebec.ca/medias/nou	Tue, 09 Dec 202	L'article parle directement des événemen	The article discusses events in Vieux-QuÃ©
4	95	0.3438	90	0.9	HIGHL	HIGHI	ici.radio-ca	Déconstruction du Colisée de Québec : pas avant la fin de 2	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T21:1	L'article traite directement de la déconst	The article discusses the deconstruction of t
5	95	0.9923	90	0.9	HIGHL	HIGHI	ici.radio-ca	Moins d'Américains, mais une année touristique record à Qu	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T21:4	L'article traite directement des performan	The article discusses the tourism situation i
6	95	0.3038	90	0.9	HIGHL	HIGHI	www.journ	Voir Québec du 31e étage: l'accès sera gratuit à l'Observato	https://www.journaldequebec.com/202	2025-12-09T11:2	L'article parle directement de l'Observato	The article discusses a new viewpoint in Qu
7	95	0.3038	90	0.9	HIGHL	HIGHI	www.journ	Voir Québec du 31e étage: l'accès sera gratuit à l'Observato	https://www.journaldequebec.com/202	2025-12-09T11:2	L'article parle directement de l'Observato	The article discusses a new attraction in Qu
8	95	0.44	90	0.9	HIGHL	HIGHI	news.goog	Voir Québec du 31e étage: l'accès sera gratuit à l'Observato	https://www.journaldequebec.com/202	Tue, 09 Dec 202	L'article parle de l'accès gratuit à l'Obser	The article discusses a new viewpoint in Qu
9	95	0.44	90	0.9	HIGHL	HIGHI	news.goog	Voir Québec du 31e étage: l'accès sera gratuit à l'Observato	https://www.journaldequebec.com/202	Tue, 09 Dec 202	L'article parle de l'accès gratuit à l'Obser	The article discusses a new attraction in Qu
10	95	0.1396	90	0.9	HIGHL	HIGHI	www.lesol	Brunch et déjeuner: un nouveau resto s'installe dans Limoilo	https://www.lesoleil.com/vivre/dans-l-a	2025-12-09T23:0	L'article parle d'un nouveau restaurant à l	The article discusses the opening of a new r
11	90	0.1833	90	0.9	HIGHL	HIGHI	news.goog	Québec | Où auront lieu les travaux du tramway en 2026? - M	https://mabeauce.com/quebec-ou-auro	Tue, 09 Dec 202	L'article traite des travaux du tramway à	The article discusses the construction of the
12	90	0.3489	90	0.9	HIGHL	HIGHI	www.journ	Don majeur de 2 M$ au Musée national des beaux-arts du Q	https://www.journaldequebec.com/202	2025-12-09T14:4	L'article concerne un don significatif au M	The article discusses a significant donation
13	90	0.0911	90	0.9	HIGHL	HIGHI	www.journ	Le Sentier de La Cache de nouveau accessible aux motonei	https://www.journaldequebec.com/202	2025-12-09T00:0	L'article traite de l'accessibilité d'un sent	The article discusses the accessibility of Le
14	85	0.9994	90	0.95	MODE	HIGHI	news.goog	WestJet's Quebec City-Montego Bay Inaugural Flight Showc	https://www.travelandtourworld.com/ne	Tue, 09 Dec 202	L'article parle d'un vol inaugural reliant Qu	The article discusses the inaugural flight fro
15	50	0.4691	90	0.9	MODE	HIGHI	www.journ	18 M$ sur 3 ans: le village historique Val-Jalbert presse Qué	https://www.journaldequebec.com/202	2025-12-09T15:4	L'article traite d'un village historique qui,	The article discusses a significant investme
16	30	0.2285	90	0.9	LOW F	HIGHI	www.narcit	Club Med va ouvrir un nouveau tout-inclus « inspiré des alpe	https://www.narcity.com/fr/club-med-to	2025-12-09T15:3	L'article concerne l'ouverture d'un nouvea	The article discusses the opening of a new a
17	10	0.0698	90	0.9	LOW_	HIGHI	ici.radio-ca	La microbrasserie 11 Comtés sur le point d'être vendue	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T05:0	L'article traite d'une microbrasserie situé	The article discusses a microbrewery, which
18	10	0.304	90	0.9	LOW_	HIGHI	ici.radio-ca	Un 20e Marché des saveurs qui comble les producteurs du E	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T18:3	L'article parle d'un événement dans le Ba	The article discusses the 20th edition of the
19	10	0.3082	90	0.9	LOW F	HIGHI	ici.radio-ca	5 km de sentiers supplémentaires au Bois-Beckett inauguré	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T19:5	L'article concerne l'inauguration de senti	The article discusses the addition of 5 km of
20	10	0.3475	90	0.9	LOW F	HIGHI	ici.radio-ca	Ouverture de la halte-douceur au centre-ville de Trois-Rivière	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T21:1	L'article concerne principalement Trois-R	The article discusses the opening of a new f
21	10	0.3703	90	0.9	LOW F	HIGHI	www.24he	Un Club Med prévoit ouvrir ses portes aux skieurs du Mont-T	https://www.24heures.ca/2025/12/08/c	2025-12-08T21:2	L'article parle d'un Club Med à Mont-Trer	The article discusses the opening of a Club
22	10	0.3717	90	0.9	LOW F	HIGHI	www.journ	Un Club Med prévoit ouvrir ses portes aux skieurs du Mont-T	https://www.journaldequebec.com/202	2025-12-08T21:2	L'article parle d'un projet à Mont-Trembla	The article discusses the opening of a Club
23	95	0.3528	85	0.9	HIGHL	MODI	www.journ	Colisée de Québec: pas de démolition avant 2027	https://www.journaldequebec.com/202	2025-12-09T16:0	L'article traite directement du Colisée de	The article discusses the ColisÃ©e de QuÃ
24	90	0.8001	85	0.9	HIGHL	MODI	news.goog	Le Groupe ALMACO Expands in Quebec City to Support Da	https://maritime-executive.com/corpora	Tue, 09 Dec 202	L'article traite de l'expansion d'une entre	The article discusses the expansion of Grou
25	90	0.6018	85	0.9	HIGHL	MODI	www.lapre	Québec Saison, un air de vacances	https://www.lapresse.ca/gourmand/res	2025-12-09T16:0	L'article traite directement de la ville de (	The article discusses the culinary scene in
26	90	0.215	80	0.9	HIGHL	MODI	www.lesol	Pas de démolition du Colisée avant 2027	https://www.lesoleil.com/actualites/act	2025-12-09T20:2	L'article traite directement du Colisée de	The article discusses the timeline for the de
27	90	0.8414	70	0.8	HIGHL	MODI	www.lesol	Des patinoires au sort incertain à Québec	https://www.lesoleil.com/actualites/act	2025-12-09T21:4	L'article traite directement des patinoires	The article discusses ice rinks in Quebec Ci
28	50	0.075	70	0.8	MODE	MODI	www.lesol	La petite révolution du LIB dépanneur	https://www.lesoleil.com/magazines/le	2025-12-09T13:4	L'article traite d'un dépanneur qui pourrai	The article discusses a local business, LIB
29	50	0.3332	70	0.8	MODE	MODI	www.lesol	Les riches jardins de Versailles… au Québec	https://www.lesoleil.com/vivre/2025/12/	2025-12-09T09:1	L'article parle d'un événement au Québec	The article discusses the gardens of Versail
30	90	0.4554	50	0.7	HIGHL	MODI	news.goog	«Mon passage au Port de Québec teste ma patience», avou	https://www.fm93.com/audio/742982/n	Tue, 09 Dec 202	L'article traite directement du Port de Qu	The article discusses an experience at the F
31	90	0.2177	50	0.7	HIGHL	MODI	www.journ	En concert à Québec: Flo Rida s'amène au Centre Vidéotron	https://www.journaldequebec.com/202	2025-12-09T13:2	L'article parle d'un concert à Québec, ce	The article discusses a concert by Flo Rida
32	85	0.9788	50	0.7	MODE	MODI	news.goog	'We have to do better,' says Quebec City mayor regarding pr	https://ca.news.yahoo.com/better-says	Tue, 09 Dec 202	L'article traite des politiques d'immigratio	The article features a statement from the Qu
33	85	0.9561	50	0.7	MODE	MODI	news.goog	'We have to do better,' says Quebec City mayor regarding pr	https://www.cbc.ca/news/canada/mont	Tue, 09 Dec 202	L'article traite des politiques d'immigratio	The article features a statement from the Qu
34	75	0.2909	50	0.7	MODE	MODI	ici.radio-ca	Bruno Marchand critique la fin du PEQ et prend la défense d	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T16:4	L'article traite de la position du maire Bru	The article discusses Bruno Marchand's crit
35	70	0.2591	50	0.7	MODE	MODI	ici.radio-ca	La CAQ priorise-t-elle le 3e lien aux dépens du pont de Québ	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T09:0	L'article traite des priorités de la CAQ co	The article discusses the CAQ's prioritizatio
36	70	0.2591	50	0.8	MODE	MODI	ici.radio-ca	La CAQ priorise-t-elle le 3e lien aux dépens du pont de Québ	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T09:0	L'article traite des priorités de la CAQ co	The article discusses the CAQ's prioritizatio

Figure 12: BigQuery output of the relevance scoring stage with conditional formatting (relevant articles in pink agentic order)



relevance_score_file1	tourism_quebec_normalized_score	relevance_score_file2	confidence_score	category_model	category_golden	source	title	url	date	reasoning_file1	reasoning_file2
50	0.1852	50	0.7	MODERATE	MODERATE	news.google.com	Immobilier: Royal LePage prévoit des hausses de prix de 7 %	https://www.lesoleil.com/affaires/2025/	Tue, 09 Dec 2025	L'article traite des prévisions immobilières	The article discusses real estate predictions
10	0.1331	50	0.7	LOW FALSE	LOW FALSE	www.journaldequebec.com	Petites patinoires olympiques: la LNH s'attend à beaucoup de	https://www.journaldequebec.com/202	2025-12-08T20:30:00	L'article traite de la LNH et des patinoires	The article discusses Olympic-related small
10	0.028	50	0.7	LOW_MODERATE	MODERATE	www.lapresse.ca	Noël sucré Rouler sa bûche sans embûche	https://www.lapresse.ca/gourmand/202	2025-12-09T11:00:00	L'article parle de Noël en général sans m	The article discusses a culinary event relatec
0	0.0658	50	0.7	NOT_FALSE	MODERATE	ici.radio-canada.ca	L'essor des arts et de la culture ne se dément pas à Tracadi	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T22:20:00	L'article traite des arts et de la culture à	The article discusses the growth of arts and
0	0.0233	50	0.7	NOT_FALSE	MODERATE	www.lesoleil.com	Carrefour, Carmila, Unlimitail et JCDecaux s'allient pour acc	https://www.lesoleil.com/communique	2025-12-09T16:40:00	L'article traite du développement du retai	The article discusses a partnership involving
10	0.0375	20	0.7	LOW FALSE	LOW FALSE	ici.radio-canada.ca	La LNH « prudemment optimiste » au sujet de l'aréna olymp	https://ici.radio-canada.ca/sports/2212	2025-12-09T13:10:00	L'article traite des Jeux Olympiques et d	The article discusses the NHL's cautious opt
95	0.3611	10	0.2	HIGHLY_LIKELY	LOW FALSE	news.google.com	Le petit Guide hivernal Impact Campus : Où patiner à Québe	https://impactcampus.ca/sports/le-peti	Tue, 09 Dec 2025	L'article traite des activités de patinage à	The article discusses winter skating location
95	0.8858	10	0.5	HIGHLY_LIKELY	LOW FALSE	news.google.com	REVIEW: Soweto Gospel Choir spreads peace in Quebec C	https://www.qctonline.com/review-sowe	Tue, 09 Dec 2025	L'article traite d'un événement culturel se	The article discusses a performance by the S
90	0.0424	10	0.7	HIGHLY_LIKELY	LOW FALSE	www.journaldequebec.com	Football: une très grosse prise pour le Rouge et Or	https://www.journaldequebec.com/202	2025-12-09T12:40:00	L'article conceme directement l'équipe de	The article discusses a football team, which
90	0.0466	10	0.7	HIGHLY_LIKELY	LOW FALSE	www.lapresse.ca	La coquette du quartier La biscuiterie Viau sur deux niveaux	https://www.lapresse.ca/maison/immo	2025-12-09T12:00:00	L'article parle d'une biscuiterie située dan	The article mentions a local business, but it
90	0.9977	10	0.5	HIGHLY_LIKELY	LOW FALSE	news.google.com	Ottawa Joins Quebec City, Niagara Falls, Montreal, Gatinea	https://www.travelandtourworld.com/ne	Tue, 09 Dec 2025	L'article parle directement de Québec et	The article mentions Quebec City alongside
85	0.3047	10	0.2	MODERATE	LOW FALSE	news.google.com	Alan Lake reçoit le Prix du CALQ - Artiste de l'année dans l	http://www.carrefourdequebec.com/cm	Tue, 09 Dec 2025	L'article parle d'un artiste de l'année dan	The article discusses an artist receiving an a
85	0.1567	10	0.7	MODERATE	LOW FALSE	news.google.com	RTC pulls plug on transition to electric buses - The Quebec	https://www.qctonline.com/rtc-pulls-plu	Tue, 09 Dec 2025	L'article traite de la transition des autobu	The article discusses the RTC's decision reg
70	0.2927	10	0.7	MODERATE	LOW FALSE	ici.radio-canada.ca	Les pistes de ski du Mont Farlagne ouvriront dès vendredi	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T17:30:00	L'article concerne l'ouverture des pistes d	The article discusses ski slopes at Mont Far
70	0.1625	10	0.3	MODERATE	LOW FALSE	www.24heures.ca	Air Transat annule 18 vols et sème la confusion: les passag	https://www.24heures.ca/2025/12/09/w	2025-12-09T16:10:00	L'annulation des vols d'Air Transat peut a	The article discusses flight cancellations by
70	0.033	10	0.3	MODERATE	LOW FALSE	www.journaldequebec.com	Célébration 2026: des prestations d'Isabelle Boulay, d'Arian	https://www.journaldequebec.com/202	2025-12-09T10:10:00	L'article parle d'un événement musical qu	The article mentions a performance by Isabel
70	0.0707	10	0.5	MODERATE	LOW FALSE	www.journaldequebec.com	L'Avalanche se rapproche d'un rare exploit	https://www.journaldequebec.com/202	2025-12-09T06:50:00	L'article parle de l'équipe de hockey de l'	The article title suggests it is about a sports
70	0.1608	10	0.2	MODERATE	LOW FALSE	www.journaldequebec.com	Air Transat annule 18 vols et sème la confusion: les passag	https://www.journaldequebec.com/202	2025-12-09T16:10:00	L'annulation de vols d'Air Transat peut aff	The article discusses flight cancellations by
70	0.9971	10	0.2	MODERATE	LOW FALSE	news.google.com	Ontario Joins British Columbia, Quebec, Alberta, Nova Scoti	https://www.travelandtourworld.com/ne	Tue, 09 Dec 2025	L'article traite de l'impact des hausses d	The article mentions Quebec but does not pr
60	0.0421	10	0.2	MODERATE	LOW FALSE	ici.radio-canada.ca	Projet de constitution : l'UMQ dénonce un « recul majeur » p	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T09:00:00	L'article traite d'un projet de loi qui pourr	The article discusses a constitutional project
50	0.1792	10	0.2	MODERATE	LOW FALSE	news.google.com	Anne-Marie Proulx reçoit le Prix du CALQ - Artiste de l'anné	http://www.carrefourdequebec.com/cm	Tue, 09 Dec 2025	L'article parle d'une artiste de Chaudière-	The article discusses an artist receiving an a
50	0.187	10	0.2	MODERATE	LOW FALSE	ici.radio-canada.ca	Minéraux critiques : le Québec et le Royaume-Uni signent un	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T02:10:00	L'accord sur les minéraux critiques peut	The article discusses critical minerals in Que
50	0.0551	10	0.3	MODERATE	LOW FALSE	ici.radio-canada.ca	La transmission familiale se poursuit à la Fromagerie St-Lau	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T17:50:00	L'article parle d'une fromagerie qui pourra	The article discusses family transmission at
50	0.05	10	0.5	MODERATE	LOW FALSE	ici.radio-canada.ca	Fonderie Horne : Legault prêt à accorder un délai, la balle es	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T20:50:00	L'article traite d'une situation qui pourrait	The article discusses a delay related to Fonc
50	0.2793	10	0.7	MODERATE	LOW FALSE	ici.radio-canada.ca	Air Transat continue d'annuler des vols à la veille de la grève	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T21:00:00	L'article traite des annulations de vols d'A	The article discusses flight cancellations by
50	0.1945	10	0.7	MODERATE	LOW FALSE	ici.radio-canada.ca	Projet Dumont Nickel : le milieu des affaires interpelle Québ	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T21:40:00	L'article traite d'un projet qui pourrait avoi	The article discusses a business project rela
50	0.2158	10	0.7	MODERATE	LOW FALSE	www.journaldequebec.com	Prévisions 2026 de Royal LePage: le prix des propriétés en	https://www.journaldequebec.com/202	2025-12-09T10:20:00	L'article traite des prévisions immobilière	The article discusses real estate price foreca
50	0.0107	10	0.7	MODERATE	LOW FALSE	www.journaldequebec.com	Un entrepreneur abonné aux contrats publics arrêté par l'UPI	https://www.journaldequebec.com/202	2025-12-09T00:00:00	L'article traite d'un entrepreneur impliqué	The article discusses a public contractor who
50	0.0638	10	0.2	MODERATE	LOW FALSE	www.lapresse.ca	Grande entreuve – Véronique Proulx, PDG de la FCCQ Des	https://www.lapresse.ca/affaires/2025-	2025-12-09T07:00:00	L'article traite de la situation économique	The article discusses VÃ©ronique Proulx, CE
50	0.0436	10	0.2	MODERATE	LOW FALSE	www.lesoleil.com	L'IA au service de l'emploi et de la compétitivité québécoise	https://www.lesoleil.com/magazines/le	2025-12-09T16:30:00	L'article aborde des sujets liés à l'emploi	The article discusses AI in relation to employ
50	0.056	10	0.7	MODERATE	LOW FALSE	www.narcity.com	Grève chez Air Transat : Voici TOUS les vols annulés jusqu'	https://www.narcity.com/fr/vols-annule	2025-12-09T13:40:00	L'article traite des annulations de vols qu	The article discusses a strike at Air Transat
50	0.9868	10	0.2	MODERATE	LOW FALSE	news.google.com	Ontario Joins British Columbia, Alberta, Quebec, Manitoba,	https://www.travelandtourworld.com/ne	Mon, 08 Dec 2025	L'article mentionne le Québec dans le cc	The article mentions Quebec but does not pr
30	0.7041	10	0.3	LOW FALSE	LOW FALSE	www.narcity.com	Ces 8 petites villes au Québec se transforment en jolis « Wi	https://www.narcity.com/fr/petites-ville	2025-12-09T20:10:00	L'article parle de petites villes au Québec	The article discusses small towns in Quebec
20	0.3125	10	0.2	LOW FALSE	LOW FALSE	news.google.com	Quebec constitution fails to recognize anglo, Indigenous min	https://ca.news.yahoo.com/quebec-co	Tue, 09 Dec 2025	L'article traite de la constitution du Québ	The article discusses the Quebec constitutio
20	0.0368	10	0.3	LOW FALSE	LOW FALSE	ici.radio-canada.ca	Quand l'art autochtone du perlage allie tradition et satire soc	https://ici.radio-canada.ca/nouvelle/22	2025-12-09T12:10:00	L'article traite de l'art autochtone sans m	The article discusses Indigenous beadwork a

Figure 13: BigQuery output of the relevance scoring stage with conditional formatting (mid-relevance articles)



relevance_score_file1	tourism_quebec_normalized_score	relevance_score_file2	confidence_score	category_model	category_golden	source	title	url	date	reasoning_file1	reasoning_file2
0	0.0084	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Les albums à surveiller en 2025	https://www.lesoleil.com/arts/musique	2025-12-09T22:00:00	L'article traite de la musique sans lien	The article discusses music albums to watch
0	0.0438	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Ce ne sont pas que les chairs qui s'affaissent avec le temps	https://www.lesoleil.com/chroniques/da	2025-12-09T09:00:00	L'article ne traite pas de sujets liés à la v	The article title does not indicate any connec
0	0.025	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Après la fraude, la méchanceté	https://www.lesoleil.com/chroniques/pa	2025-12-09T19:50:00	L'article ne traite pas de sujets liés à la v	The article title suggests a focus on issues o
0	0.0123	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Ageas annonce le succès du placement d'EUR 450 millions	https://www.lesoleil.com/communique	2025-12-09T17:10:00	L'article concerne un placement financ	The article discusses a financial placement t
0	0.0094	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Aramis Group - Mise en œuvre d'un programme de rachat d'	https://www.lesoleil.com/communique	2025-12-09T17:20:00	L'article concerne un programme de ra	The article appears to be a press release abo
0	0.0171	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Avis aux médias : MADD Canada dévoilera un nouveau pann	https://www.lesoleil.com/communique	2025-12-09T21:30:00	L'article concerne un événement à St. Cé	The article appears to be a press release fron
0	0.014	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Caisse Française de Financement Local EMTN 2025-19	https://www.lesoleil.com/communique	2025-12-09T13:10:00	L'article ne mentionne pas Québec ou Ca	The article title and URL suggest it pertains t
0	0.0169	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	COURTOIS SA : Variation droits de vote 11/2025	https://www.lesoleil.com/communique	2025-12-09T16:20:00	L'article ne mentionne pas Québec ou la	The article title suggests a focus on voting rig
0	0.0097	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	EDF : Nomination de Béatrice Bigois au Comité Exécutif du	https://www.lesoleil.com/communique	2025-12-09T17:00:00	L'article concerne une nomination au s	The article discusses the nomination of BÃ©
0	0.0121	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	EssilorLuxottica: Nombre total de droits de vote et d'actions	https://www.lesoleil.com/communique	2025-12-09T17:00:00	L'article concerne des informations fin	The article is a press release about EssilorLu
0	0.0278	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Globex annonce les premiers résultats de forage sur Rouyn-	https://www.lesoleil.com/communique	2025-12-09T14:00:00	L'article concerne des résultats de forag	The article appears to be a press release rela
0	0.0247	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Hermès International : Actions et droits de vote au 30 novem	https://www.lesoleil.com/communique	2025-12-09T17:00:00	L'article concerne des informations fin	The article appears to focus on HermÃ¨s Inte
0	0.0108	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	IPSEN - Achats effectués dans les conditions de l'Article 5 d	https://www.lesoleil.com/communique	2025-12-09T17:00:00	L'article ne concerne pas la ville de Qu	The article title and URL do not provide any ir
0	0.0084	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Ipsos – Evolution de l'organisation	https://www.lesoleil.com/communique	2025-12-09T17:30:00	L'article ne mentionne pas Québec ou la	The article appears to be a press release regi
0	0.0166	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	JCDecaux : Déclaration des transactions sur actions propres	https://www.lesoleil.com/communique	2025-12-09T15:20:00	L'article traite de transactions d'action	The article discusses stock transactions relat
0	0.0184	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	La Société de Gestion AGF Limitée déclare un dividende por	https://www.lesoleil.com/communique	2025-12-09T12:30:00	L'article concerne une déclaration de d	The article appears to focus on a corporate ac
0	0.0295	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	L'application EXIT Life Ready : Planifiez aujourd'hui pour un	https://www.lesoleil.com/communique	2025-12-09T17:10:00	L'article ne mentionne pas Québec ou la	The article title does not provide any indicatio
0	0.0137	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Le Greater Bay Area Science Forum 2025 s'ouvre à Guangz	https://www.lesoleil.com/communique	2025-12-09T19:50:00	L'article traite d'un forum scientifique à	The article pertains to the Greater Bay Area S
0	0.0359	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Monument Mining annonce les résultats de son Assemblée	https://www.lesoleil.com/communique	2025-12-09T16:30:00	L'article concerne une entreprise minière	The article discusses the results of a mining
0	0.0249	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	SAFE - RESULTATS SEMESTRIELS 2025	https://www.lesoleil.com/communique	2025-12-09T17:00:00	L'article ne mentionne pas Québec ou la	The article title and URL suggest it is a press
0	0.0075	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	SUSE et Evroc unissent leurs forces pour accélérer la souve	https://www.lesoleil.com/communique	2025-12-09T08:00:00	L'article traite de la souveraineté numéric	The article does not mention Quebec City or
0	0.0376	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Télésat conclut un partenariat stratégique avec le gouverner	https://www.lesoleil.com/communique	2025-12-09T14:10:00	L'article traite de partenariats militaires	The article appears to be a press release abo
0	0.0114	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Valeo - Déclaration des droits de vote - Novembre 2025	https://www.lesoleil.com/communique	2025-12-09T13:30:00	L'article ne traite pas de Québec ou de	The article appears to be a press release regi
0	0.0125	0	0	NOT_FALSE	NOT_FALSE	www.lesoleil.com	VINCI : Déclaration des transactions sur actions propres Du	https://www.lesoleil.com/communique	2025-12-09T16:40:00	L'article concerne des transactions fin	The article is about transactions on shares re
0	0.0077	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	La prochaine phase du cessez-le-feu impossible sans press	https://www.lesoleil.com/monde/2025/	2025-12-09T17:40:00	L'article traite de sujets internationaux	The article appears to discuss an internationa
0	0.0047	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Le ministère de la Justice peut lever les scellés sur l'affaire	https://www.lesoleil.com/monde/2025/	2025-12-09T17:30:00	L'article traite d'une affaire judiciaire im	The article discusses a legal matter related to
0	0.008	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Le président ukrainien refuse de céder des territoires à la Ru	https://www.lesoleil.com/monde/2025/	2025-12-09T22:40:00	L'article traite d'un sujet international s	The article discusses international political m
0	0.0228	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	La caricature de Côté	https://www.lesoleil.com/opinions/caric	2025-12-09T09:00:00	L'article ne traite pas de sujets liés à l	The article appears to focus on a caricature a
0	0.0226	0	1	NOT_FALSE	NOT_FALSE	www.lesoleil.com	Auger-Aliassime et Mboko nommés joueurs de l'année au p	https://www.lesoleil.com/sports/2025/12	2025-12-09T15:30:00	L'article traite de joueurs de tennis cana	The article discusses sports players and thei
0	0.0154	0	1	NOT_FALSE	NOT_FALSE	www.narcity.com	These Ontario grocery stores were ranked as the 'cheapest'	https://www.narcity.com/cheapest-onta	2025-12-09T18:10:00	L'article traite des épiceries en Ontario s	The article discusses grocery stores in Ontar
0	0.0059	0	1	NOT_FALSE	NOT_FALSE	www.narcity.com	11 items à moins de 30 $ sur Amazon qui font de beaux cac	https://www.narcity.com/fr/amazon-ide	2025-12-09T21:00:00	L'article concerne des idées de cadea	The article discusses items available on Ama
0	0.0431	0	1	NOT_FALSE	NOT_FALSE	www.narcity.com	Pornhub dévoile les recherches les plus fréquentes au Cana	https://www.narcity.com/fr/pornhub-rec	2025-12-09T15:00:00	L'article traite de recherches sur un site	The article discusses search trends on a por
0	0.1074	0	1	NOT_FALSE	NOT_FALSE	www.narcity.com	This German-inspired Christmas village near Toronto has sto	https://www.narcity.com/german-christ	2025-12-09T01:30:00	L'article parle d'un village de Noël en Ont	The article discusses a Christmas village nea
0	0.0663	0	1	NOT_FALSE	NOT_FALSE	www.narcity.com	This small town 1 hour from Toronto has starred in dozens o	https://www.narcity.com/ontario-christ	2025-12-09T01:00:00	L'article traite d'une ville près de Toronto	The article discusses a small town near Toro
0	0.0365	0	1	NOT_FALSE	NOT_FALSE	www.narcity.com	Toronto drivers can be fined $615 for breaking this rule and th	https://www.narcity.com/toronto/toront	2025-12-09T22:10:00	L'article concerne des lois de conduite à	The article discusses driving fines in Toronto,

Figure 14: BigQuery output of the relevance scoring stage with conditional formatting (filtered-out low relevance articles, all 3 models were good at filtering out noise)

Figure 15: Login challenge page encountered during scraping

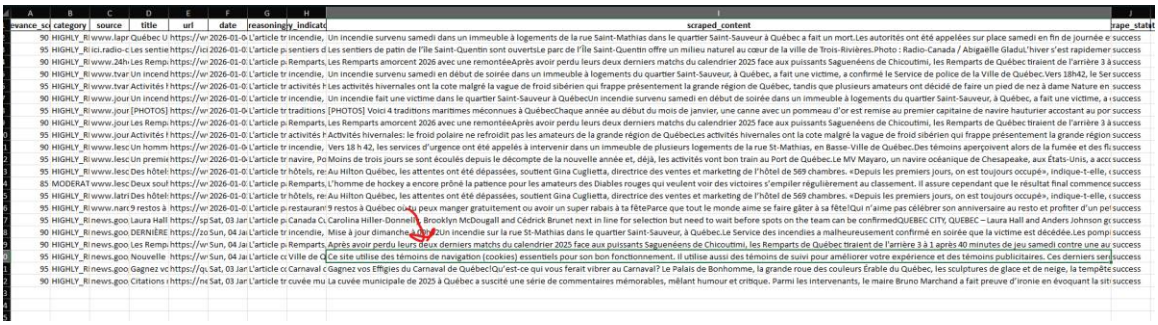


Figure 16: Cookie consent banner polluting scraped content

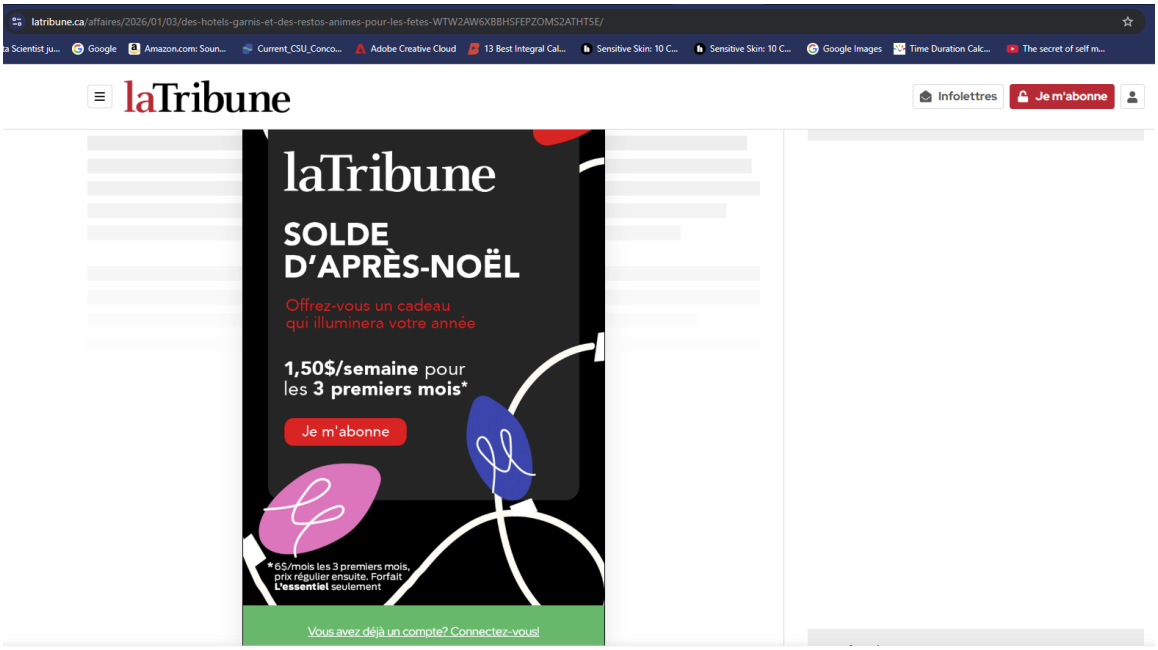


Figure 17: Paywall restriction on La Tribune requiring payment

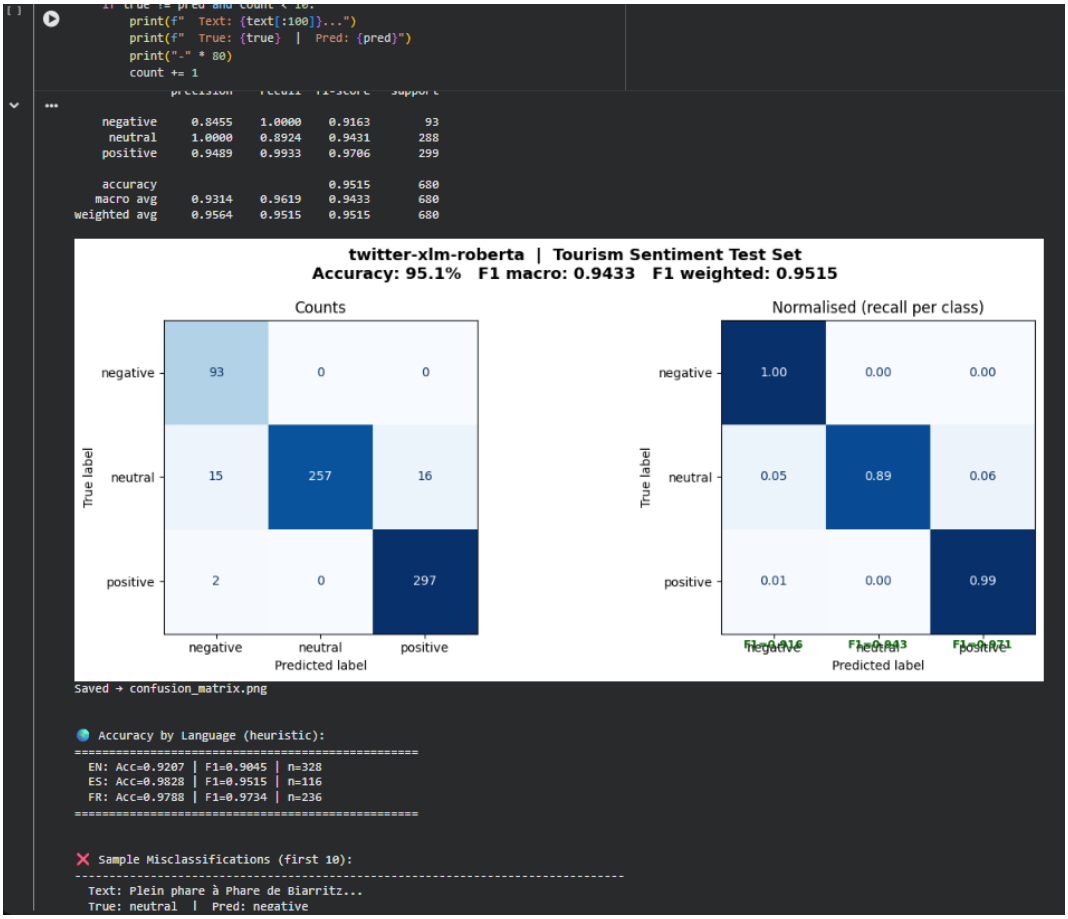


Figure 18: Twitter-XLM-RoBERTa sentiment classification confusion matrix on the tourism test set

```
print(f""True: {true} -> Pred: {pred} (conf. {conf:.2f})")
print("-" * 90)
```

CHALLENGE LEADERBOARD - 100 Tourism Samples (EN+FR)

Model	Acc	F1	EN	FR	Sarc	Medium	Hard	⏱
🥇 ClapAI Multilingual	55.0%	53.8%	60.0%	50.0%	38.9%	55.6%	54.7%	21s
🥈 Tabularisai Multilingual	54.0%	52.2%	52.0%	56.0%	33.3%	80.6%	39.1%	3s
🥉 Twitter XLM-RoBERTa	49.0%	49.4%	56.0%	42.0%	16.7%	69.4%	37.5%	5s
CardiffNLP XLM-R Multilingual	49.0%	48.1%	50.0%	48.0%	16.7%	58.3%	43.8%	9s
NLPtown BERT (1-5 stars)	42.0%	42.7%	40.0%	44.0%	22.2%	41.7%	42.2%	5s

Figure 19: Sentiment model challenge leaderboard across multilingual models on tourism samples

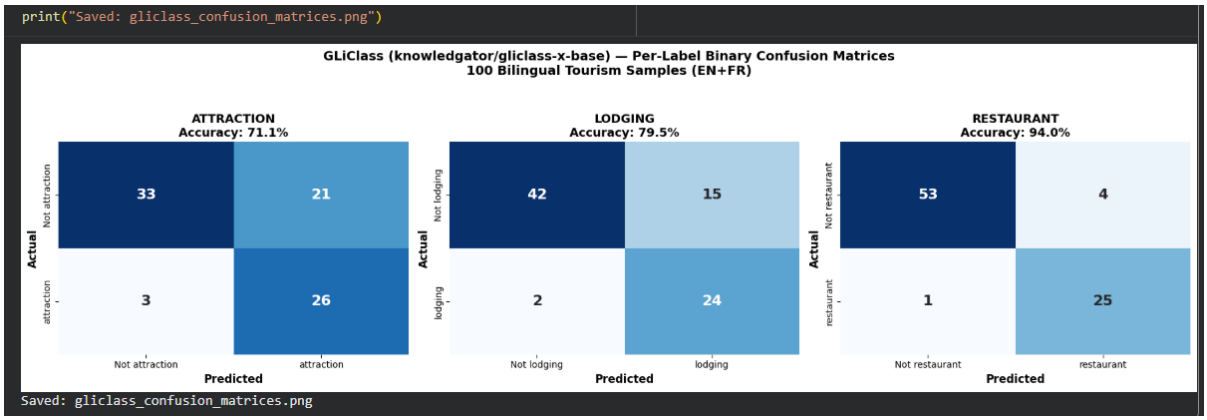


Figure 20: GLiClass zero-shot per-label confusion matrices on the synthetic bilingual tourism dataset

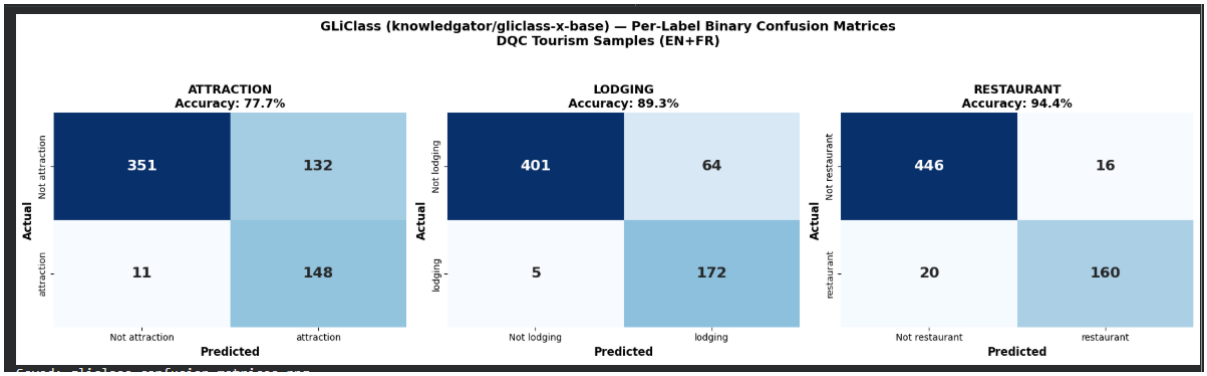


Figure 21: GLiClass zero-shot per-label confusion matrices on real DQc tourism articles

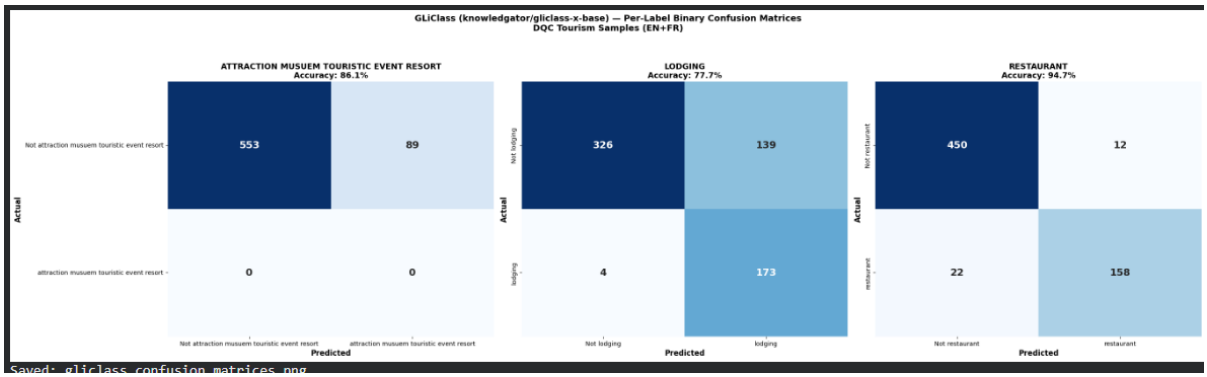


Figure 22: GLiClass zero-shot confusion matrices with expanded attraction labels (alternative view)

```
[18]
✓ 0s
from gliclass import GLiClassModel
model = GLiClassModel.from_pretrained("knowledgator/gliclass-x-base")
print(model)

GLiClassModel(
  (model): GLiClassUniEncoder(
    (text_projector): FeaturesProjector(
      (linear_1): Linear(in_features=768, out_features=768, bias=True)
      (act): GELUActivation()
      (dropout): Dropout(p=0.1, inplace=False)
      (linear_2): Linear(in_features=768, out_features=768, bias=True)
    )
    (classes_projector): FeaturesProjector(
      (linear_1): Linear(in_features=768, out_features=768, bias=True)
      (act): GELUActivation()
      (dropout): Dropout(p=0.1, inplace=False)
      (linear_2): Linear(in_features=768, out_features=768, bias=True)
    )
    (pooler): FirstTokenPooling1D()
    (scorer): ScorerDot()
    (dropout): Dropout(p=0.1, inplace=False)
    (encoder_model): DebertaV2Model(
      (embeddings): DebertaV2Embeddings(
        (word_embeddings): Embedding(250104, 768, padding_idx=0)
        (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (encoder): DebertaV2Encoder(
        (layer): ModuleList(
          (0-11): 12 x DebertaV2Layer(
            (attention): DebertaV2Attention(
              (self): DisentangledSelfAttention(
                (query_proj): Linear(in_features=768, out_features=768, bias=True)
                (key_proj): Linear(in_features=768, out_features=768, bias=True)
                (value_proj): Linear(in_features=768, out_features=768, bias=True)
                (pos_dropout): Dropout(p=0.1, inplace=False)
                (dropout): Dropout(p=0.1, inplace=False)
              )
              (output): DebertaV2SelfOutput(
                (dense): Linear(in_features=768, out_features=768, bias=True)
                (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
                (dropout): Dropout(p=0.1, inplace=False)
              )
            )
            (intermediate): DebertaV2Intermediate(
              (dense): Linear(in_features=768, out_features=3072, bias=True)
              (intermediate_act_fn): GELUActivation()
            )
            (output): DebertaV2Output(
              (dense): Linear(in_features=3072, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
        )
        (rel_embeddings): Embedding(512, 768)
        (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
      )
    )
  )
)
```

Figure 23: Printout of the GLiClass model architecture (DeBERTa-v2 backbone with projectors), exploding the models in Python also show token limits , here 512 for example

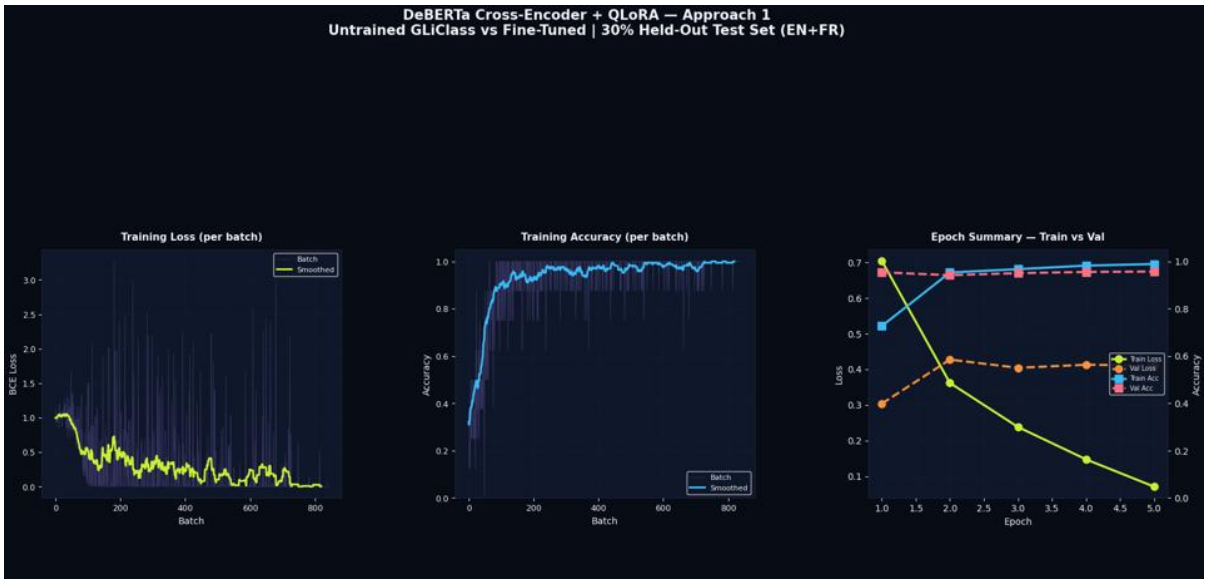


Figure 24: DeBERTa cross-encoder + QLoRA training curves for fine-tuned GLiClass

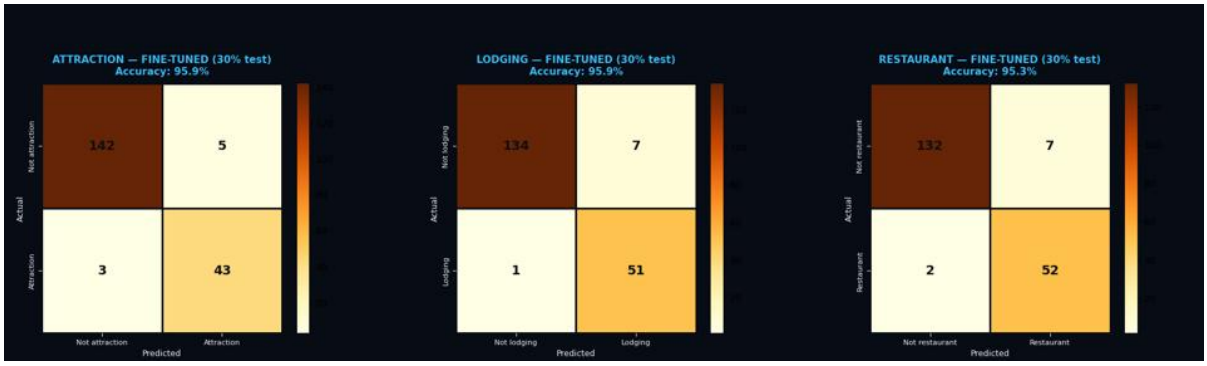


Figure 25: Fine-tuned GLiClass per-label confusion matrices on the 30% held-out test set

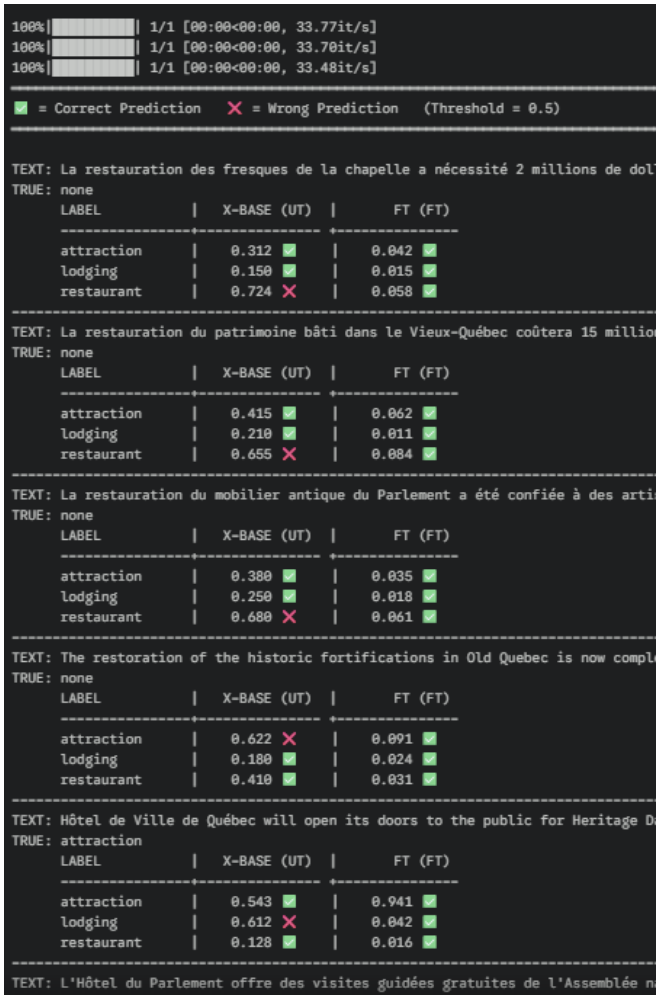


Figure 26: Per-sample predictions comparing untrained GLiClass-x-base against the fine-tuned model

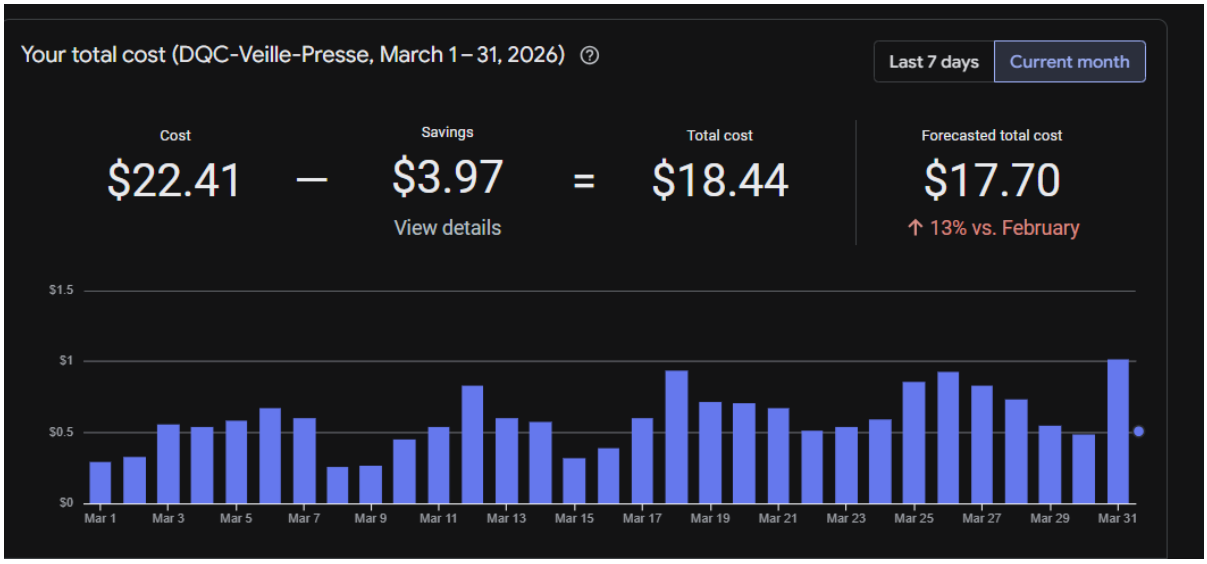


Figure 27: Monthly Google Cloud Platform cost dashboard for the DQc-Veille-Presse project

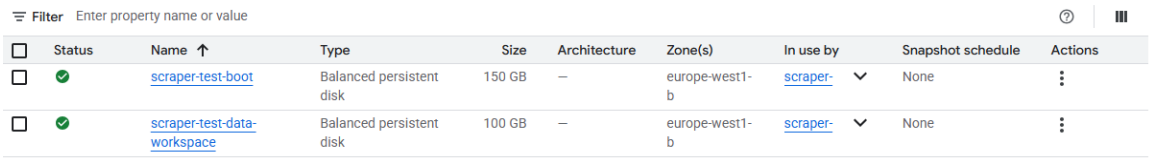


Figure 28: GCP idle storage cost problem

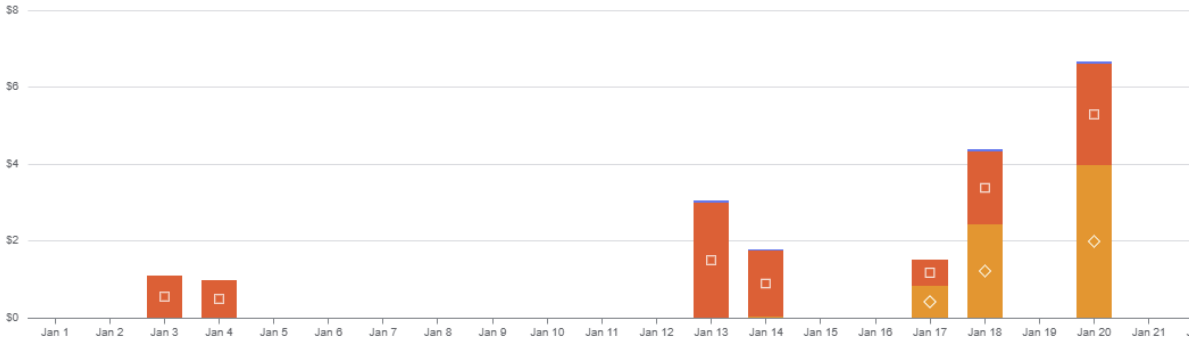


Figure 29: Daily pipeline cost spike after switching to Gemini late January

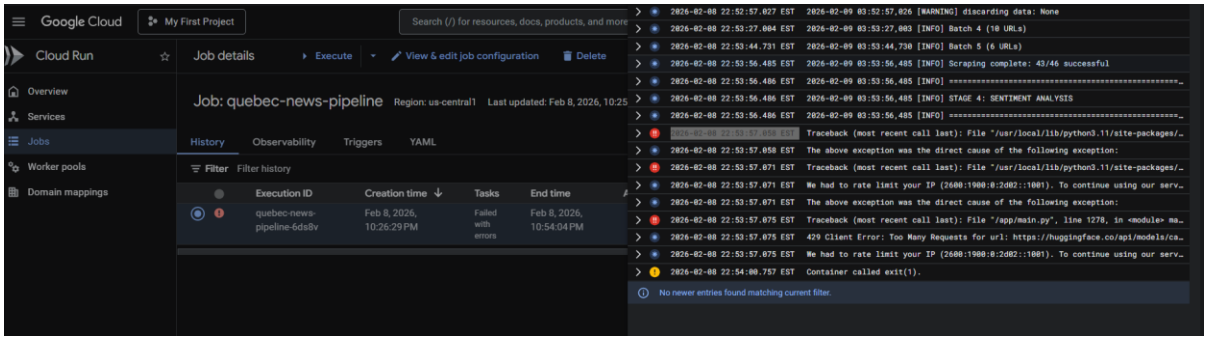


Figure 30: Cloud Run job execution history showing a failed pipeline run with traceback due to HuggingFace rate limits

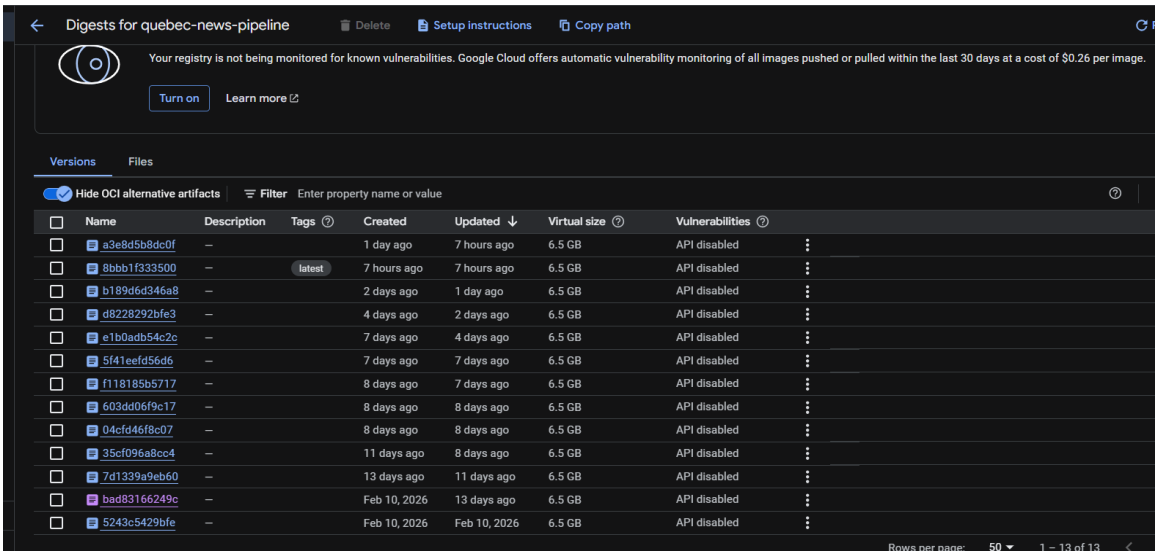


Figure 31: Docker images rebuild

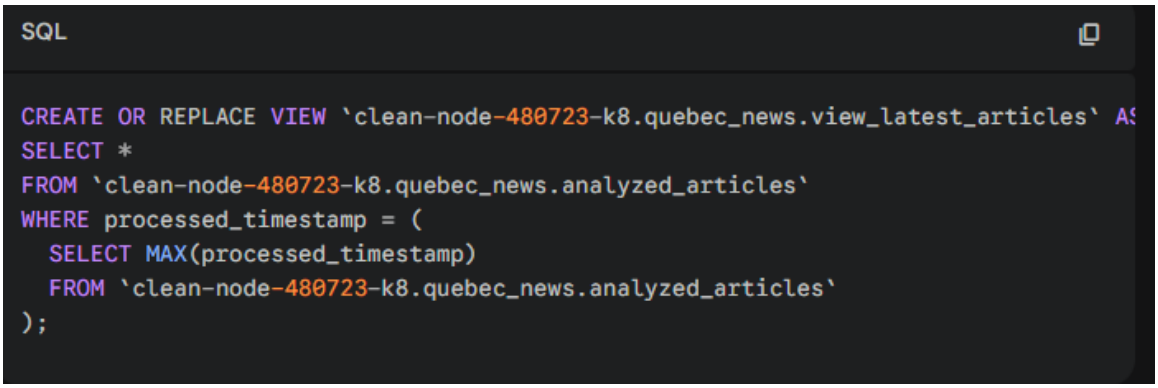


Figure 32: BigQuery historical data management view

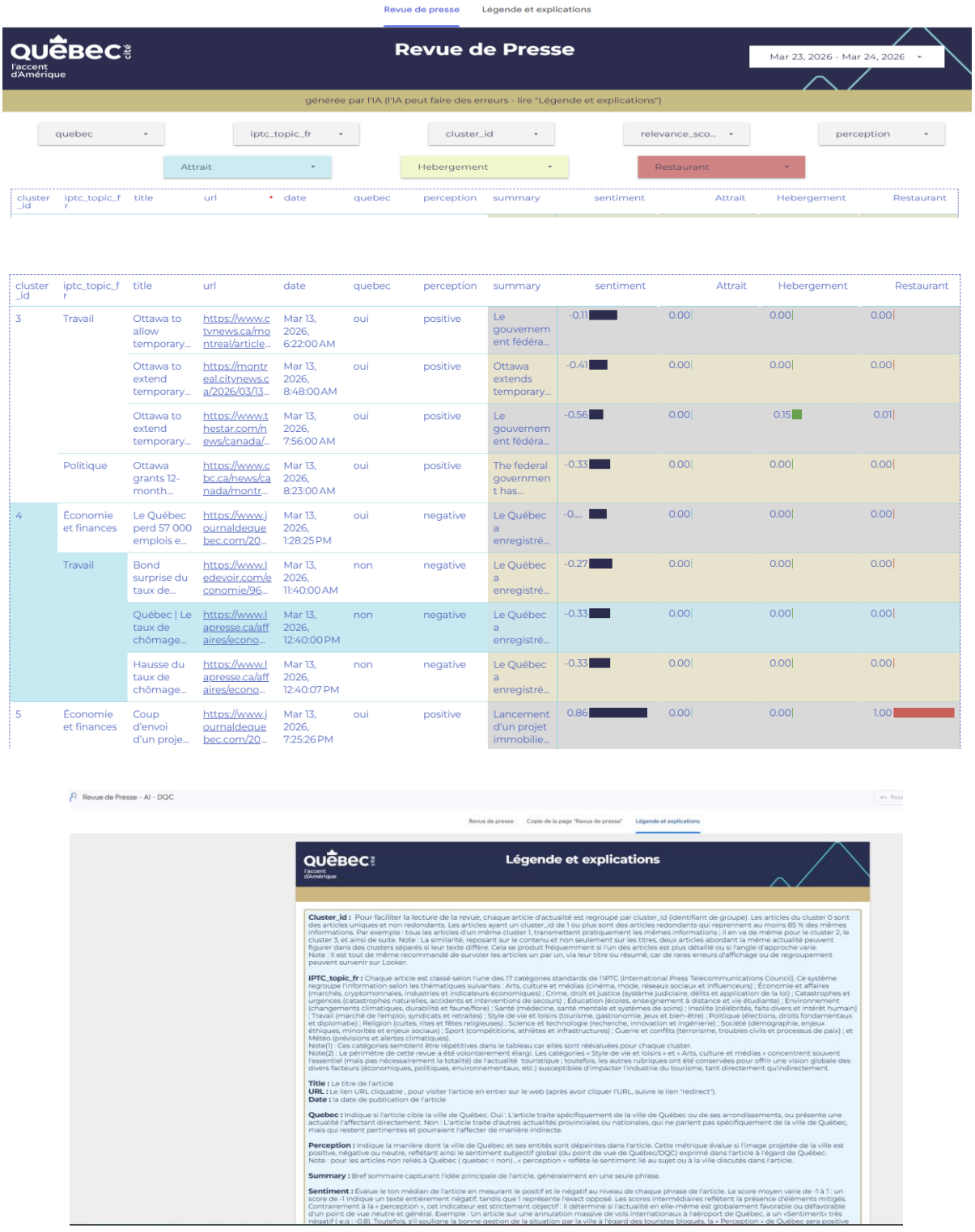


Figure 33: Final DQC Looker dashboard view of clustered articles enriched with sentiment, perception, and domain scores (as well as the dashboard's legend)

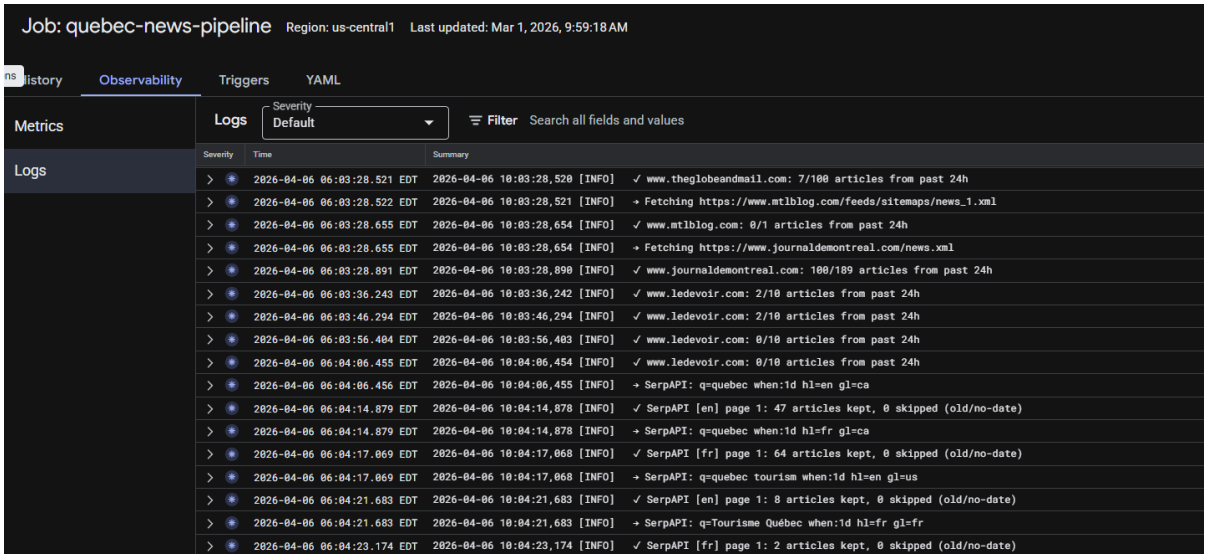


Figure 34: Cloud Run pipeline job observability logs showing per-source article counts

entity	entity_label	mentions	sentiment	sent_conf	ent_conf	clusters
bruno marchand	person	7	neutral	0.989	0.835	0,1,2,4
christian mars	person	4	neutral	0.712	0.832	0
stéphane lachance	person	3	neutral	0.983	0.834	0,4
m. marchand	person	3	neutral	0.75	0.681	0,2
olivia lexhaller	person	2	neutral	0.953	0.839	0
françois legault	person	2	neutral	0.997	0.784	0,2
patrice drouin	person	2	neutral	0.858	0.838	0
arnaud marchand	person	2	neutral	0.971	0.797	0,2

Figure 35: ABSA entity level sentiment analysis showing NER-extracted entities with sentiment scores, confidence levels, and cluster associations

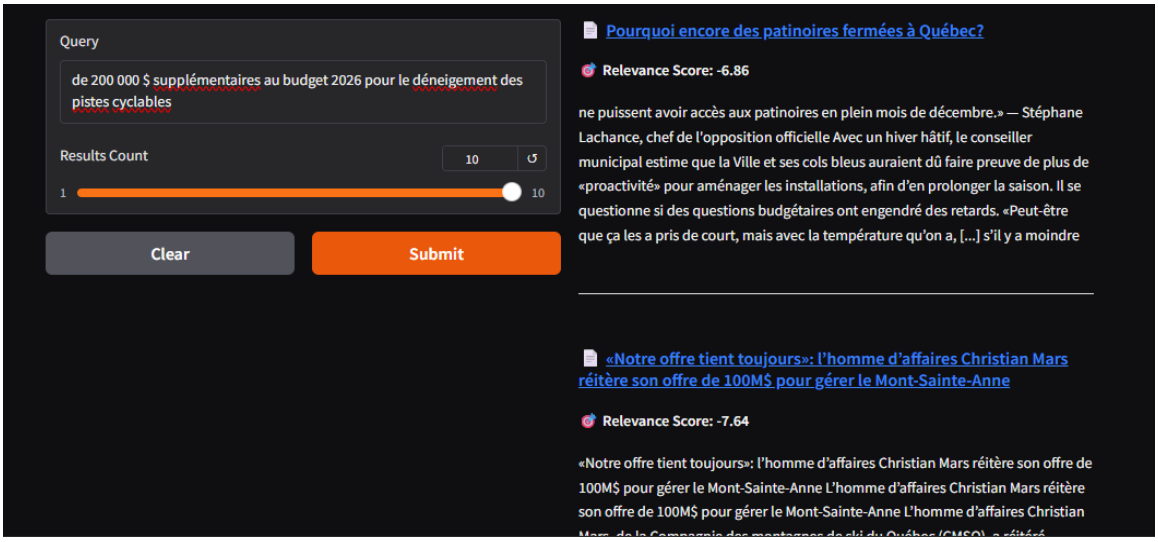


Figure 36: Embedding-only search returning limited results

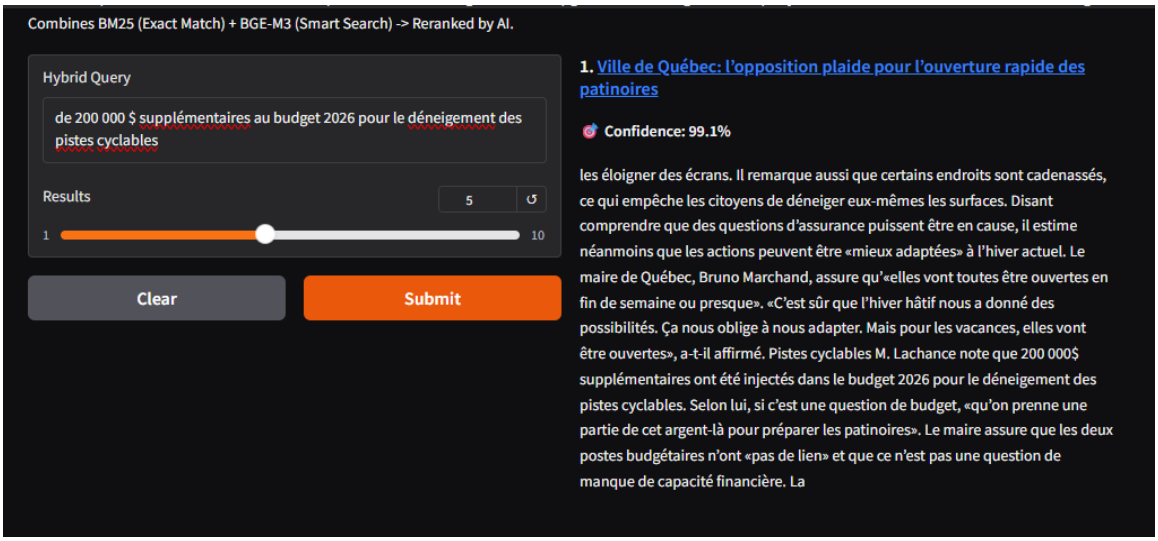


Figure 37: BM25 hybrid search implementation , you can see the pseudo-exact phrase match in query and retrieved chunk